ImGee: Avionics & Flight-Sim UI Toolkit

Comprehensive Developer & User Manual • ImGee v1.0.0

`Dear ImGui` is an immediate-mode graphical user interface library built fundamentally for game-engine debugging, tool development, and developer panels. When flight simulator plugin developers attempt to build production-grade avionics interfaces, settings windows, or interactive kneeboards with stock Dear ImGui, they immediately encounter several fundamental architectural gaps:

1. **Missing Glyphs and Font Atlas Friction:** Standard font atlases omit extended Unicode mathematical and symbol blocks (such as ↻, ▲, ▼, ✓, ✵, or ✈), resulting in missing-glyph diamond boxes (**`?`** or placeholder squares) unless heavy custom icon fonts are baked into VRAM.
2. **Volatile Tooltips:** Vanilla **`ImGui::SetTooltip()`** follows the mouse cursor frame-by-frame, continuously flickering, resetting scroll state, and clipping off-screen when hovering near flight simulator window borders.
3. **Clunky Input Affordances:** Checkboxes look like desktop form controls rather than physical cockpit switches; sliders do not respond naturally to the mouse scroll wheel; and there is no universal "revert to calibrated default" pattern for complex tuning parameters.
4. **Rigid Text Rendering:** Stock text rendering lacks inline semantic sizing, baseline alignment, and formatted Markdown tables with column resizing and persistent sorting.

`ImGee` is a specialized **ImGui toolkit** designed specifically to bridge these gaps. At its core, ImGee is an **avionics toolkit for ImGui** that extends, wraps, and simplifies everyday developer needs—from high-polish information displays to tactile flight-deck controls. Bundled as the premier package within the **`ImgWindow contrib/`** ecosystem (and architected to stand as an independent open-source library), ImGee transforms Dear ImGui into a cockpit-grade design system with zero mandatory dependencies, zero runtime allocations in hot loops, and complete backward compatibility.

```
ImgWindow/
├── contrib.h                          # Master Concentrator (-> contrib/includes.h)
└── contrib/
    ├── includes.h                     # Level-1 Container Package Switchboard
    ├── imgee.h                        # Package Forwarding Header (-> ImGee/imgee.h)
    │
    ├── ImGee/                         # The ImGee Flight-Deck ImGui Toolkit
    │   ├── imgee.h                    # Canonical ImGee Master Header & Version Contract
    │   ├── includes.h                 # Folder-level Roll-Up Header (-> imgee.h)
    │   ├── extensions.h               # Subsystem Roll-Up Header (-> extensions/includes.h)
    │   ├── widgets.h                  # Subsystem Roll-Up Header (-> widgets/includes.h)
    │   ├── imgee-global.h             # Opt-in Global Scope Aliases
    │   ├── LICENSE                    # Open-Source License (BSD 3-Clause)
    │   ├── docs/
    │   │   ├── ImGee_User_Guide.md    # Comprehensive User Manual (this document)
    │   │   ├── ImGee_User_Guide.pdf   # Compiled Publication-Grade Manual
    │   │   └── images/                # Figure assets and widget screenshots
    │   ├── extensions/                # Subsystems (Typography, Markdown, TimedTooltip)
    │   └── widgets/                   # Immediate-mode flight controls & vector icons
    │
    ├── extensions/                    # Legacy Forwarding Shims (Do Not Use for New Code)
    ├── widgets/                       # Legacy Forwarding Shims (Do Not Use for New Code)
    ├── extensions.h                   # Legacy Forwarding Shim (Do Not Use for New Code)
    └── widgets.h                      # Legacy Forwarding Shim (Do Not Use for New Code)
```

2. Structural Architecture: The Two Categories of Content

Every component within ImGee is delivered strictly as a **single, self-contained C++ header file (`.h` only)**. There are zero `.cpp` compilation units to add to your build system, zero static or dynamic link dependencies, and zero order-of-inclusion restrictions. Each header contains its own header guards and self-contained forward declarations; you can `#include` ImGee headers in any order without circular dependency hazards.

ImGee separates its code into two distinct architectural categories:

2.1 Flagship Extensions (`extensions/`)

Extensions are feature-complete, heavyweight subsystems that solve deep architectural limitations of stock Dear ImGui. Unlike simple widgets, extensions maintain internal or frame-to-frame state, manage timing delays, parse complex syntax trees, or manage font registries.

ImGee provides **three flagship extensions** that form the core value of the package:

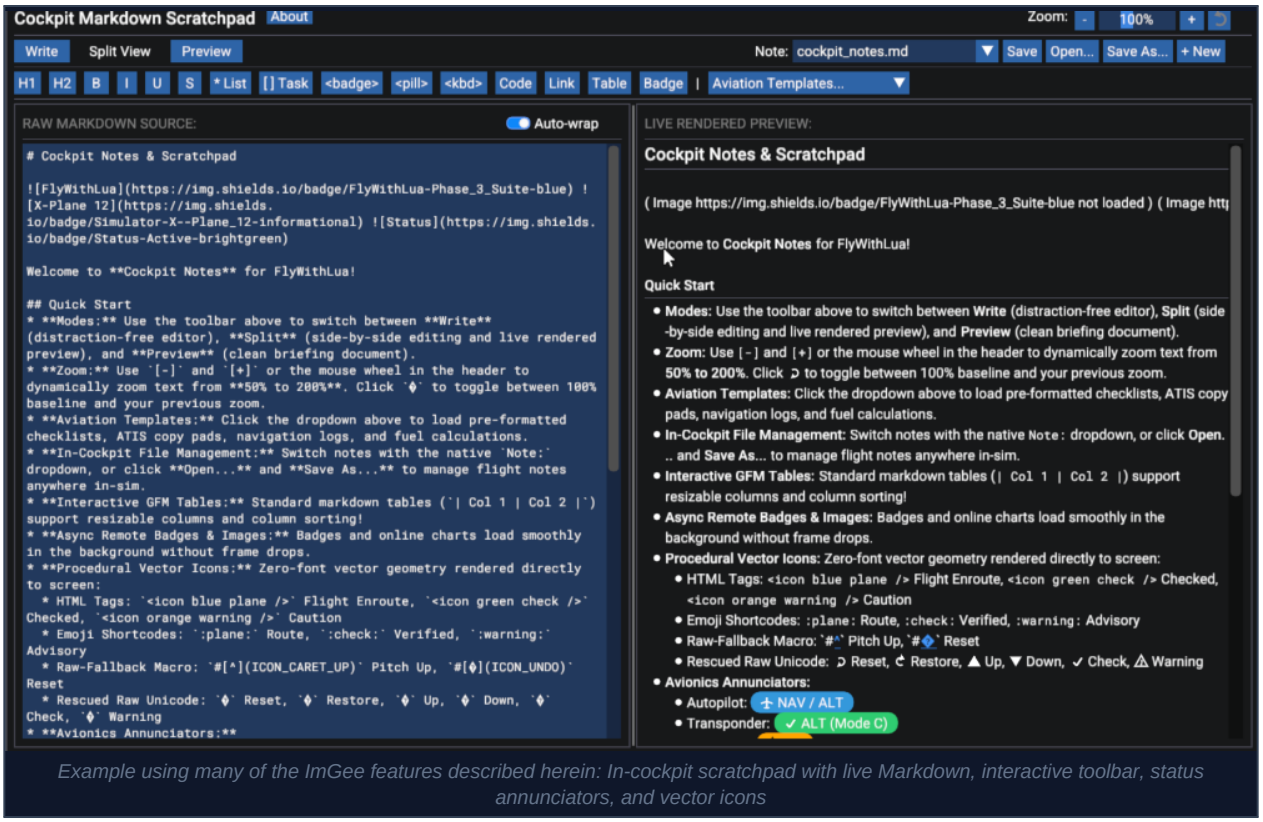
- **`TimedTooltip` (`imgui_tooltip_ext.h`):** Replaces volatile, cursor-following vanilla `ImGui::SetTooltip()` with stationary, delay-timed tooltips featuring intelligent display boundary clamping, cursor avoidance, and low-water scroll dismissal.
- **`Markdown` (`imgui_markdown_ext.h`):** Replaces flat plain ASCII with interactive CommonMark, full GFM tables with tristate column sorting and drag-resizing, parameterized styling tags (`<badge>`, `<pill>`, `<icon>`), and custom vector button faces (`ButtonMD`).
- **`Typography` (`imgui_typography_ext.h`):** An 8-slot semantic font role registry (`ImFontRole`), RAII scoped font guards, and the `StyledText` engine for inline baseline-aligned sizing with zero Markdown document overhead.

> **IMPORTANT**
>
> **The Flagship Core:** We consider `TimedTooltip`, `Markdown`, and `Typography` the **flagship foundation** of the entire ImGee suite. We strongly believe every flight simulation plugin should use these modern subsystems instead of the stock or open-source versions from which they derive.

2.2 Cockpit & Avionics Widgets (`widgets/`)

Widgets are lightweight, zero-dependency, immediate-mode flight-deck controls (`imgui_knobs.h`, `imgui_toggle_buttons.h`, `imgui_mouse_wheel_controls.h`, `imgui_badges.h`, `imgui_spinners.h`, etc.). They hook directly into Dear ImGui's ambient context (`GImGui`, ID stack, `ImGuiStorage`, and active window draw lists) and live directly in the root `ImGui::` namespace to feel completely native.

- **Completely Independent:** Even if a project chooses not to install or enable any of the three extensions, all widgets in `widgets/` remain 100% usable and self-sufficient.
- **Zero Mandatory State:** Widgets execute strictly within the immediate-mode paradigm, drawing antialiased vector geometry directly to `ImDrawList`.



Example using many of the ImGee features described herein: In-cockpit scratchpad with live Markdown, interactive toolbar, status annunciators, and vector icons

3. Common Design Standards & Ecosystem Philosophy

ImGee follows strict architectural patterns established by the broader Dear ImGui community:

3.1 Omar Cornut's Ecosystem Standard (Single Point of Discovery)

In official Dear ImGui extensions and ecosystem projects (such as **`implot`**, **`imgui_club`**, and **`imnodes`**), Omar Cornut explicitly advocates extending **`namespace ImGui { ... }`**.

- **Discoverability & Autocomplete:** Modern C++ developers rely heavily on IDE autocomplete. When typing **`ImGui::`**, developers immediately discover every capability available across the toolkit in one place.
- **Zero Cognitive Overhead:** Developers are not forced to memorize artificial root namespaces (such as **`extgui::`** or **`imgee::`**) or manage scattered **`using namespace`** directives.

3.2 Second-Level Sub-Namespaces for Subsystems

While extending **`namespace ImGui`** is essential, placing heavy packages directly into the root risks colliding with future Dear ImGui core functions. ImGee cleanly encapsulates its extensions within **second-level sub-namespaces**:

- **`ImGui::TimedTooltip::`**
- **`ImGui::Markdown::`** (with official shorthand alias **`ImGui::MD::`**)
- **`ImGui::Typography::`**

Sub-namespaces allow extensions to reuse Dear ImGui's standard vocabulary without ambiguity:

```cpp
ImGui::TimedTooltip::Text("Hover-delayed stationary tooltip.");
ImGui::MD::TextWrapped("Configure your **avionics** below.");
```

3.3 Convenient Root Shorthands & Global Scope

- Shorthands like `ImGui::TextMD(...)` and `ImGui::ButtonMD(...)` are exposed in the root `ImGui::` namespace.
- Defining `#define IMGEE_ENABLE_GLOBAL_NAMESPACES` publishes `Markdown::`, `MD::`, and `TimedTooltip::` directly to the global scope for maximum brevity.

3.4 Working with Immediate-Mode Widgets in C++

Developers consuming ImGee widgets should expect standard Dear ImGui immediate-mode conventions:

- **Caller-Owned State:** Controls accept pointers to caller-owned primitives (e.g. `float* value`, `bool* open`). ImGee never allocates heap memory for persistent values unless explicitly requested.
- **Ambient Context:** Functions implicitly draw into the currently active `ImGuiWindow` and utilize Dear ImGui's ambient style and font stacks.
- **ID Stack Safety:** Push and Pop IDs (`ImGui::PushID` / `ImGui::PopID`) apply seamlessly across all ImGee controls, rotary knobs, and reset buttons.

3.5 The ImGee UX Design Manifesto (Human Factors Standard)

For the full architectural philosophy, ergonomics theory, and interaction rules governing ImGee, see the companion document:

- The ImGee UX Design Manifesto (`ImGee_UX_Design_Manifesto.pdf`):
- **Flight Deck Cognitive Reality:** Protecting Expert Mode (muscle memory & peripheral vision); eliminating Thinking Mode interruptions.
- **The Zero-Inert Law:** Mandatory active cursor affordances (`Hand`, `ResizeNS`, `ResizeEW`) for all interactive elements.
- **First-Gesture Discoverability:** Natural list ordering (+1 on scroll down) and eliminating initial-gesture false negatives.
- **Monotonic vs. Categorical Dialectic:** Strict rules for clamping (`wrap = false`) vs. circular wrapping (`wrap = true`).
- **The Lens UI Principle:** Modal focus latching with > 6 px hysteresis during open-menu wheel navigation.
- **Container Pass-Through:** Closed pulldowns pass scroll events through to parent scrollable containers.
- **Reversibility & Non-Destructive Defaults:** 1-click restore (↺) with instant undo history.

---

4. Generic Dear ImGui Core & Multi-Language Bindings

While developed and battle-tested within the X-Plane flight simulation ecosystem, **ImGee core is 100% generic Dear ImGui**. Nothing in the C++ toolkit inherently requires X-Plane or any specific host environment. Any application, game engine, or simulator utilizing Dear ImGui can consume ImGee directly.

4.1 LuiGee: Lua Bindings for FlyWithLua

For Lua developers, ImGee's flight-deck capabilities are brought directly into the Lua scripting environment as **LuiGee**.

- **Broad Simulator Support:** Available for ImGui-based windows in FlyWithLua across all X-Plane versions from **X-Plane 11.20 through the latest X-Plane 12+ releases and into the future**.
- **Zero-Boilerplate Scripting:** Exposes cockpit switches, knobs, vector icons, and timed tooltips to Lua developers with idiomatic syntax.

4.2 Community & Other Language Bindings

We are actively considering bindings for other environments and programming languages (such as **Python**, **Rust**, and **C#**). If developers in the wider Dear ImGui ecosystem are interested in bringing ImGee to other platforms or developing language wrappers, we warmly welcome community discussions and collaboration!

---

5. Integration Models & Include Paths

One of the central design principles of this **ImGui toolkit** is immediacy. You do not need to initialize subsystems, configure factories, or register draw callbacks. Everything is configured and ready by default.

5.1 Why "contrib.h"? (ImgWindow Ecosystem Context)

In this distribution, **ImGee is provided alongside `ImgWindow v2` as a premier community contribution** housed inside the `contrib/` directory. This architectural relationship explains why the top-level all-in-one concentrator header is named `contrib.h`.

5.2 Submodule Include Paths: Recommended Front Door

In production environments, developers rarely clone `ImgWindow` directly as a standalone project; instead, they integrate it as a **Git submodule** (e.g., in `third-party/ImgWindow/` or `libs/ImgWindow/`). Standard project build rules (CMake, Makefiles, Xcode, Visual Studio) typically add the directory *containing* `ImgWindow` to the compiler's include search paths (`-I`), rather than adding the inner `ImgWindow` repository root directly.

Because of this standard submodule topology, typing `#include "contrib.h"` will fail in most host projects unless the build system specifically includes the inner repository folder.

The safest, most standard, and recommended all-in-one inclusion is:

```cpp
#include "ImgWindow/contrib.h"
```

(Note: If your build system explicitly adds the `ImgWindow` directory directly to your `-I` compiler flags, `#include "contrib.h"` works identically).

5.3 Direct Package Integration: ImGee Suite Only

If you wish to include *only* the ImGee suite—bypassing the container switchboard or avoiding potential future third-party packages in `contrib/`—you can include ImGee directly using any of our symmetrical package headers:

```cpp
// Option A: Canonical top-level contrib forwarder (Recommended):
#include "ImgWindow/contrib/imgee.h"

// Option B: PascalCase case-insensitive alias:
#include "ImgWindow/contrib/ImGee.h"

// Option C: Inner suite master header (or folder roll-up):
#include "ImgWindow/contrib/ImGee/imgee.h"
#include "ImgWindow/contrib/ImGee/includes.h"
```

5.4 A-La-Carte Subsystem Inclusion

ImGee is architecturally decoupled. If your plugin only requires flight-deck widgets (rotary knobs, sliders, toggles) or only requires extensions (Markdown, TimedTooltip, Typography), you can include either half independently:

```cpp
// Option A: Include only extensions (TimedTooltip, Markdown, Typography):
#include "ImgWindow/contrib/ImGee/extensions.h"

// Option B: Include only cockpit widgets (Knobs, Toggles, Wheel Sliders, Badges, Icons):
#include "ImgWindow/contrib/ImGee/widgets.h"
```

NOTE

> **Legacy Backward-Compatibility Shims:** The headers `contrib/extensions.h` and `contrib/widgets.h` exist quietly solely to prevent breaking code written against older experimental builds. They forward into `ImGee/extensions/` and `ImGee/widgets/`, respectively. New code should always use the canonical `ImgWindow/contrib/ImGee/...` paths above.

5.5 Standalone / Independent Submodule Usage

When ImGee is spun out into its own dedicated open-source Git repository and submoduled directly into a host application:

```cpp
// Direct root include:
#include "imgee.h"

// Or via submodule path:
#include "ImGee/imgee.h"
```

5.6 Ready by Default Out of the Box

Every capability described in this manual is **100% active and enabled out of the box**:

- **Procedural Vector Icons** automatically resolve without font atlas baking.
- **Stationary Timed Tooltips** eliminate cursor jitter and support rich Markdown out-of-the-box.
- **Cockpit Input Affordances** (sliders with reset buttons, rotary knobs, animated toggles) compile cleanly into `ImGui::`.
- **Formatted Markdown & GFM Tables** render interactive documentation with zero external dependencies.

(For projects with strict binary constraints or symbol conflicts, granular opt-outs are provided in Appendix A).

---

6. Orientation Roadmap & User Manual Directory

To guide developers through the suite, this manual is structured into dedicated, modular reference sections:

- **Part I: Flagship Extensions Suite**
- TimedTooltip Extension: Stationary delayed tooltips, screen-edge clamping, and scroll dismissal.
- Markdown Extension: CommonMark rendering, GFM tables with tristate sorting and drag-resizing, and custom tags.
- Typography Extension: `ImFontRole` 8-slot registry, scoped font guards, and baseline-aligned `StyledText`.
- **Part II: Cockpit Widgets Catalog**
- Procedural Vector Icons: Zero-texture vector engine and `IconButton` / `ButtonIcon`.
- Avionics Rotary Knobs: 270° potentiometers with ticks, readout, and 1-click reset.
- Mouse-Wheel Sliders with Reset: Sliders with natural wheel stepping and 1-click undo chasers.
- Status Badges & Pills: Badges, pills, annunciators, and `<kbd>` keycaps.
- Tactile Animated Toggles: Cockpit rocker switches with dual-state tooltips.
- Radar Spinners & Countdown Dials: Activity spinners and approach countdown timers.
- Directional Popups & SelectPulldown: Directional blossoming popups, headroom queries, and 1-line dataref selectors with Lens UI.
- **Part III: Reference & Standards**
- Appendix A: Preprocessor Opt-Out Configuration: Granular switches for binary size and symbol isolation.
- **Companion Standard:** The ImGee UX Design Manifesto (`ImGee_UX_Design_Manifesto.pdf`): Canonical human factors and interaction engineering tenets.

---

7. Flagship Extension: Stationary Timed Tooltips (`imgui_tooltip_ext.h`)

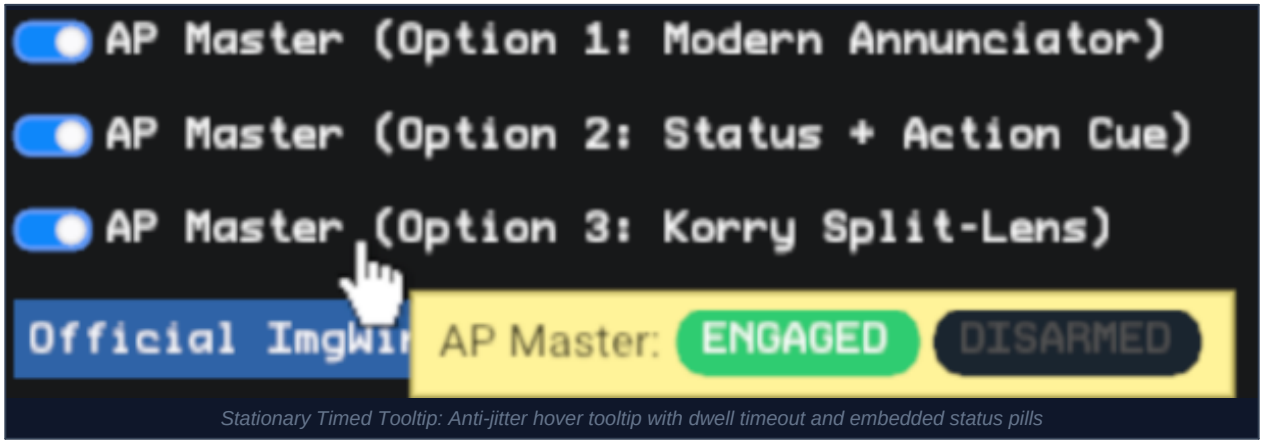
Vanilla tooltips follow the cursor frame-by-frame like a bee, creating visual distraction, resetting scroll state, and clipping off-screen when near monitor borders. ImGee elevates Dear ImGui tooltips to be stationary, anchored, and cursor-aware:

```cpp
// 1. Plain text stationary tooltip
ImGui::Button("Flaps 15");
ImGui::TimedTooltip::Text("Takeoff flap detent. Generates additional lift with moderate drag.");

// 2. Full Markdown stationary tooltip
ImGui::Button("GPWS");
ImGui::TimedTooltip::TextMD(
    "### Ground Proximity Warning System\n"
    "* **Terrain Ahead:** Pull up immediately.\n"
    "* **Glideslope:** Deviation exceeds 1.3 dots."
);
```

Key Tooltip Affordances:

- **Initial Dwell Delay:** Prevents accidental popup flashing while sweeping the mouse across busy cockpit panels.
- **Stationary Anchor:** Freezes in position once visible, allowing effortless, relaxed reading.
- **Smart Screen Clamping:** Automatically nudges inside the simulator or host window boundaries.
- **Wheel-Defeat Dismissal:** Rolling the mouse wheel immediately closes the tooltip so it never blocks scrolling views.
- **Font Scaling & Damped Compression:** Automatically dampens font zoom (using a non-linear curve with a 0.90f floor and 1.65f ceiling) so tooltips stay readable without ballooning into unwieldy screen-filling rectangles on ultra-wide or 4K monitors.



Stationary Timed Tooltip: Anti-jitter hover tooltip with dwell timeout and embedded status pills

8. Flagship Extension: Rich Markdown Engine & GFM Tables (`imgui_markdown_ext.h`)

The ImGee Markdown engine transforms plain text strings into fully interactive cockpit documentation:

```cpp
[CPP]

const char* doc = R"(
# Flight Operations Manual
The **Autopilot Flight Director System** (AFDS) manages aircraft trajectory.

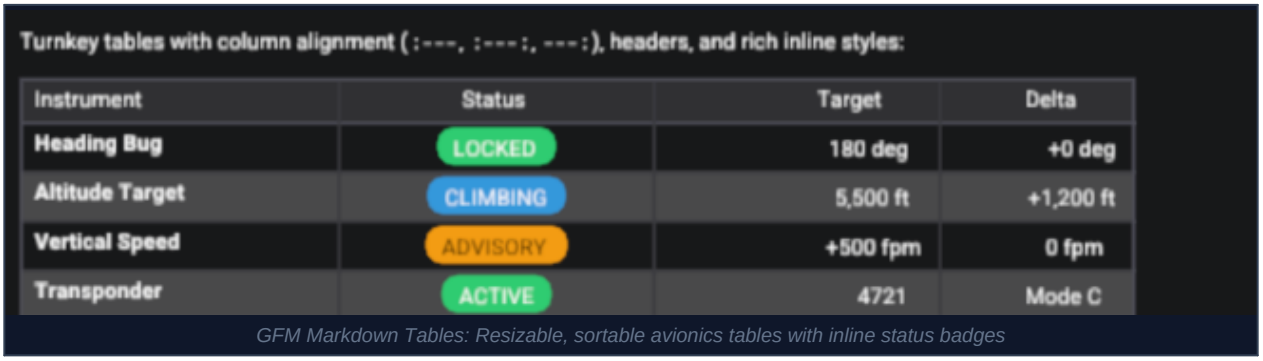
### Status Annunciations
| Mode | Status | Armed | Minimum Speed |
| :--- | :---: | :---: | :--- |
| **LNAV** | <badge-success>ENGAGED</badge-success> | Yes | `V2 + 15` |
| **VNAV** | <badge-warning>ARMED</badge-warning> | Yes | `250 KTS` |
| **ILS** | <badge>STANDBY</badge> | No | `VREF + 5` |

> [!NOTE]
> Ensure barometric altimeters are set to standard pressure (29.92 IN HG / 1013 HPA) when passing transition alt
)";

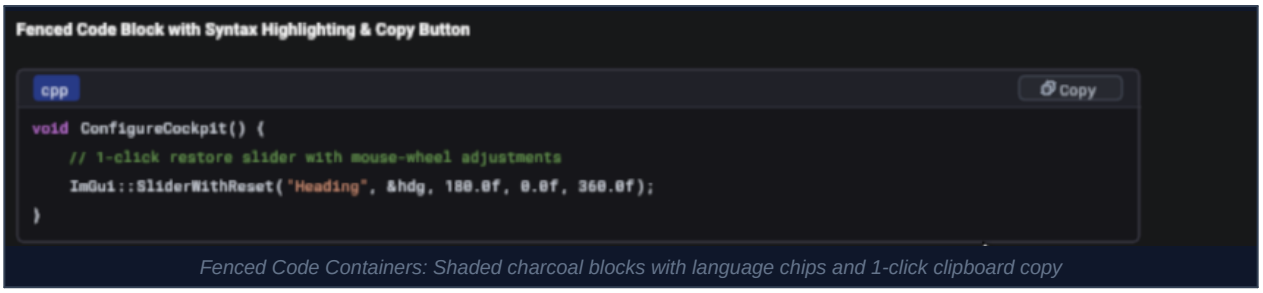
ImGui::Markdown::Text(doc);
```

Key Markdown Affordances:

- **Tristate Column Sorting:** Click any column header to cycle Ascending $\rightarrow$ Descending $\rightarrow$ Default sort.
- **Interactive Drag Sizing:** Adjust column dividers directly with instant mouse feedback.
- **Persistent Presets:** Save and restore column widths via inline comment tags (`<!-- table-state: ... -->`).
- **Code Block Copy Buttons:** Shaded code blocks automatically include one-click clipboard copying with 1.5s visual feedback.



GFM Markdown Tables: Resizable, sortable avionics tables with inline status badges



Fenced Code Containers: Shaded charcoal blocks with language chips and 1-click clipboard copy

8.1 Declarative Semantic Style Tags & Macro Aliases

Rather than writing verbose C++ code or nesting raw push/pop color commands, developers can define custom semantic Markdown tags dynamically during plugin startup:

1. Semantic Style Tags (`RegisterMarkdownStyleTag`)

Map custom tags to composite typography styles combining colors, weights, fonts, and background capsules:

```cpp
// Register declarative styles during plugin initialization:
ImGui::RegisterMarkdownStyleTag("cam", "green bold mono");
ImGui::RegisterMarkdownStyleTag("viewmode", "mono bg=#88888840");
ImGui::RegisterMarkdownStyleTag("alert", "bg=#ff4444 fg=#ffffff bold");
```

Use them seamlessly inside any Markdown text:

```cpp
ImGui::MD::Text("Set camera to <cam>Heading</cam> mode or switch to <viewmode>Circle view</viewmode>.");
```

2. Macro Template Tag Aliases (`RegisterMarkdownTagAlias`)

Register reusable tag macros that expand into arbitrary Markdown syntax with `%s` placeholder substitution:

```cpp
// Register tag templates:
ImGui::RegisterMarkdownTagAlias("opt", "<pill #444444 fg=#ffffff>%s</pill>");
ImGui::RegisterMarkdownTagAlias("callsign", "**`%s`**");
ImGui::RegisterMarkdownTagAlias("hotkey", "<kbd>%s</kbd>");
```

When evaluated, `<opt>Circle view</opt>` expands into `<pill #444444 fg=#ffffff>Circle view</pill>`, complete with background capsule geometry and baseline alignment.

3. Hex Alpha Transparency & Auto-Contrast

- **Alpha Channels:** Color specifications support 4-digit (`#RGBA`) and 8-digit (`#RRGGBBAA`) hex codes (e.g. `bg=#88888840` or `bg=#8884` for a 25% translucent charcoal pill).
- **Auto-Contrast:** In capsule tags (`<badge>`, `<pill>`), bare color tokens default to the background color (`bg_color`), and the text foreground color is automatically chosen based on perceived luminance (white text on dark capsules, dark text on bright capsules). Explicit `fg=` or `text=` overrides this auto-calculation when desired.
- **Font Role Integration:** Style tags automatically pull loaded fonts from the `ImFontRole` registry:
- `mono` / `monospace` $\rightarrow$ Slot 5 (`Monospace`) with fallback to Slot 7 (`MonospaceMedium`)
- `bold` $\rightarrow$ Slot 3 (`Bold`) with fallback to Slot 4 (`BoldLarge`)
- `italic` $\rightarrow$ Slot 6 (`Italic`) with fallback to Slot 8 (`BoldItalic`)

---

9. Foundation Extension: Semantic Typography & Font Roles (`imgui_typography_ext.h`)

Cockpit user interfaces require strict typographic hierarchy: primary instruments need large, legible digits; captions and annunciator legends require micro-typography; and datarefs require clear monospaced alignment.

ImGee provides two complementary typography systems:

1. **Semantic Font Roles (`ImFontRole`):** A 9-slot registry (Slots 0..8) linking logical design roles to loaded rasterized fonts with automatic hierarchical fallbacks.
2. **Inline Styled Text Engine (`ImGui::StyledText`):** Lightweight XML-style tag parsing with baseline alignment, formatted variadic `printf` support, and zero Markdown document overhead.

9.1 The `ImFontRole` Registry

```cpp
enum class ImFontRole {
    Regular = 0,        // Slot 0: Primary interface font (e.g. 14px)
    Title,              // Slot 1: Section headers, dialog titles (e.g. 16px)
    Small,              // Slot 2: Captions, annotations, secondary subtext (e.g. 12px)
    Bold,               // Slot 3: Strong emphasis, table headers (e.g. Bold 14px)
    BoldLarge,          // Slot 4: Hero banners, major readouts (e.g. Bold 16px)
    Monospace,          // Slot 5: Datarefs, callsigns, frequencies (e.g. Mono 14px)
    Italic,             // Slot 6: Citations, hints, metadata (e.g. Italic 14px)
    MonospaceMedium,    // Slot 7: Numeric avionics readouts, dials (e.g. Mono Medium 15px)
    BoldItalic,         // Slot 8: Strong soft emphasis (e.g. Bold-Italic 14px)
};
```

Bind loaded fonts using **`ImgFontAtlas`** or standard ImGui calls:

```cpp
ImgFontAtlas::SetFontForRole(ImFontRole::MonospaceMedium, avionicsMonoFont);
```

9.2 Inline Styled Text (`ImGui::StyledText`)

When you want rich typography without building a full CommonMark AST, **`ImGui::StyledText`** parses XML-style tags (**`<b>`**, **`<i>`**, **`<mono>`**, **`<huge>`**, **`<small>`**, **`<color>`**, **`<badge>`**, **`<pill>`**) inline with continuous baseline alignment.

It supports full **`printf`**-style formatted variadic arguments out of the box with zero runtime boilerplate:

```cpp
// 1. Variadic formatted text:
ImGui::StyledText("ALT: <huge>%d</huge> <small>%s</small> | SPD: <mono>%.1f</mono> <small>KTS</small>",
                  altitude, altUnits, airspeed);

// 2. Dynamic status pills with computed values:
ImGui::StyledText("Status: <pill #2ecc71>%s</pill> | VOR: <mono>%s</mono>",
                  statusName, identStr);

// 3. Unformatted pointer pairs:
ImGui::StyledTextUnformatted(bufStart, bufEnd);
```

---

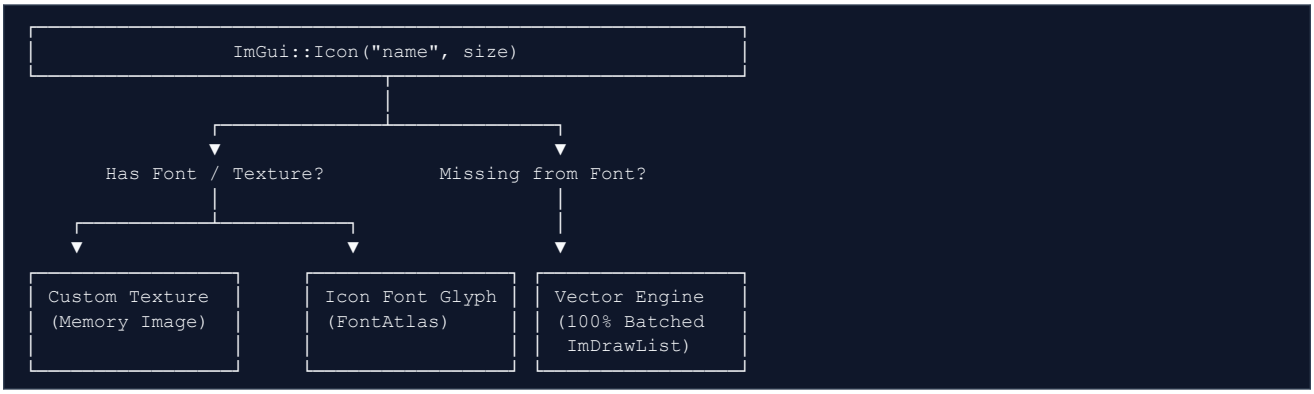
10. Cockpit & Avionics Widgets Showcase

Standard Dear ImGui controls are built for developer tools and debug menus. ImGee extends ImGui with a complete suite of physical, tactile avionics widgets designed for real cockpit ergonomics.

10.1 Procedural Vector Icon Engine & Glyphs (`imgui_icons.h`)

Standard Dear ImGui plugins often suffer from missing glyphs when attempting to display symbols like reset arrows (↺), sort carets (▲, ▼), checkmarks (✓), gears (⚙), or airplanes (✈).

ImGee includes a **zero-texture, zero-font procedural vector engine** (**`imgui_icons.h`**). It submits antialiased geometric primitives (lines, arcs, bezier curves, triangles) directly to Dear ImGui's active **`ImDrawList`**.



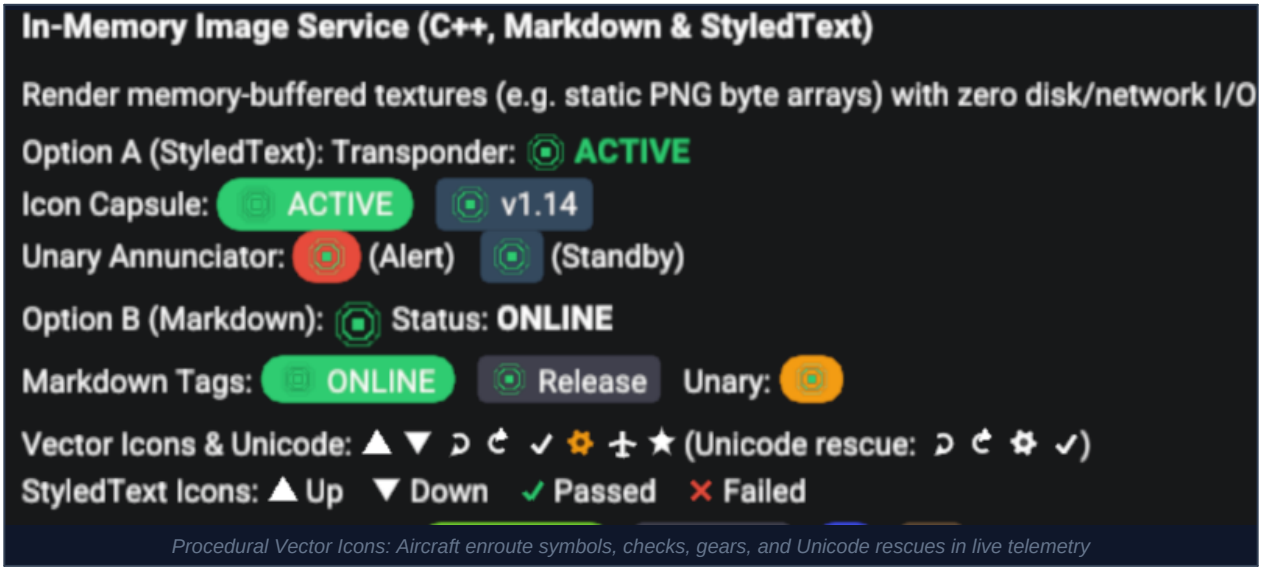
Vector Push-Buttons: `IconButton` & `ButtonIcon`

ImGee provides turnkey button helpers that render procedural vector icons without font dependencies:

```cpp
// 1. Square Icon-Only Button with native hand cursor and tooltip
if (ImGui::IconButton("##ResetHdg", "undo", ImVec2(0, 0), "Reset Heading to 360°")) {
    s_Heading = 360.0f;
}

// 2. Icon + Text Button (Icon on Left)
if (ImGui::ButtonIcon("Settings", "gear")) {
    OpenSettingsDialog();
}

// 3. Text + Icon Button (Icon on Right)
if (ImGui::ButtonIcon("Next Waypoint", "caret-right", ImVec2(160, 0), /*icon_on_right=*/true)) {
    AdvanceWaypoint();
}
```



Procedural Vector Icons: Aircraft enroute symbols, checks, gears, and Unicode rescues in live telemetry

10.2 Avionics Rotary Knobs & Dimmer Dials (`imgui_knobs.h`)

Standard sliders feel unnatural for cockpit instruments. `ImGui::Knob` implements an avionics-grade potentiometer:

```
          . - ~ ~ - .
       . '    .---.    ' .
     /       /     \       \
    :       |   *   |       :   <- 270° "Omega" Arc
     \       \     /       /       with Open Deadband at 6 o'clock
       . '    '---'    ' .
           ' -  *  - '
              OFF
```

Features:

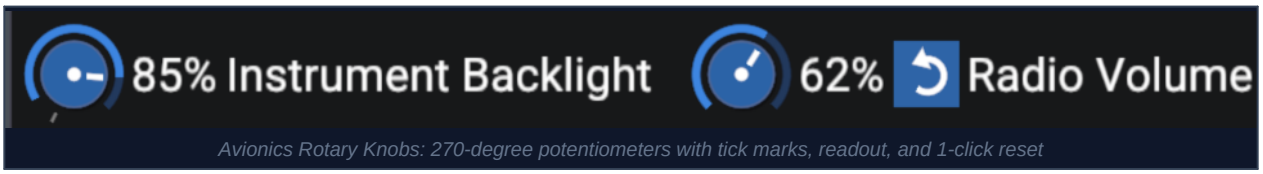
1. **270-Degree "Omega" Arc:** An open deadband at 6 o'clock immediately communicates an interactive dial.
2. **Tactile Two-Step Detente Switch:** Counter-clockwise rotation dims smoothly down to minimum operating value (e.g., 5%). A deliberate extra mouse-wheel click or drag snaps across the detente gap into the powered-down `"OFF"` state. Rotating clockwise from `"OFF"` snaps instantly back to minimum active power.
3. **Scroll-Wheel Primary:** Hover and roll the wheel with fine-tuning modifiers (`Shift` for 10x, `Alt`/`Ctrl` for 0.1x).

[CPP]

```cpp
static float s_InstrumentLight = 0.75f;

// 1. Standard Knob with perimeter ticks
ImGui::Knob("Panel Light", &s_InstrumentLight, 0.0f, 1.0f,
            ImGuiKnobFlags_ShowTicks | ImGuiKnobFlags_ValueBelow);

// 2. Composite Knob with Reset Chaser
static const float s_DefaultLight = 0.80f;
ImGui::KnobWithReset("Backlight", &s_InstrumentLight, 0.0f, 1.0f,
                     s_DefaultLight, "%.0f%%");
```



Avionics Rotary Knobs: 270-degree potentiometers with tick marks, readout, and 1-click reset

---

10.3 Mouse-Wheel Sliders & Reset Chasers (`imgui_mouse_wheel_controls.h`)

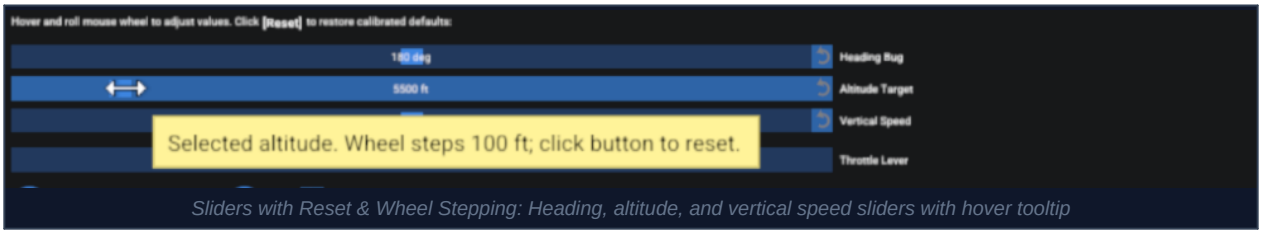
Settings tweaking in flight simulations is fraught with user anxiety: if an author tweaks an aircraft trim or graphic setting, they fear losing their calibrated value.

ImGee solves this with **Inline 1-Click Restore-to-Default Chasers**:

[CPP]

```cpp
static float s_Gamma = 2.2f;
static const float s_DefaultGamma = 2.2f;

// Renders slider with natural mouse-wheel stepping and a ⟲ button that appears
// whenever s_Gamma differs from s_DefaultGamma. Clicking restores default instantly.
ImGui::SliderWithReset("Display Gamma", &s_Gamma, 1.0f, 3.0f, s_DefaultGamma, "%.2f");
```



Sliders with Reset & Wheel Stepping: Heading, altitude, and vertical speed sliders with hover tooltip

---

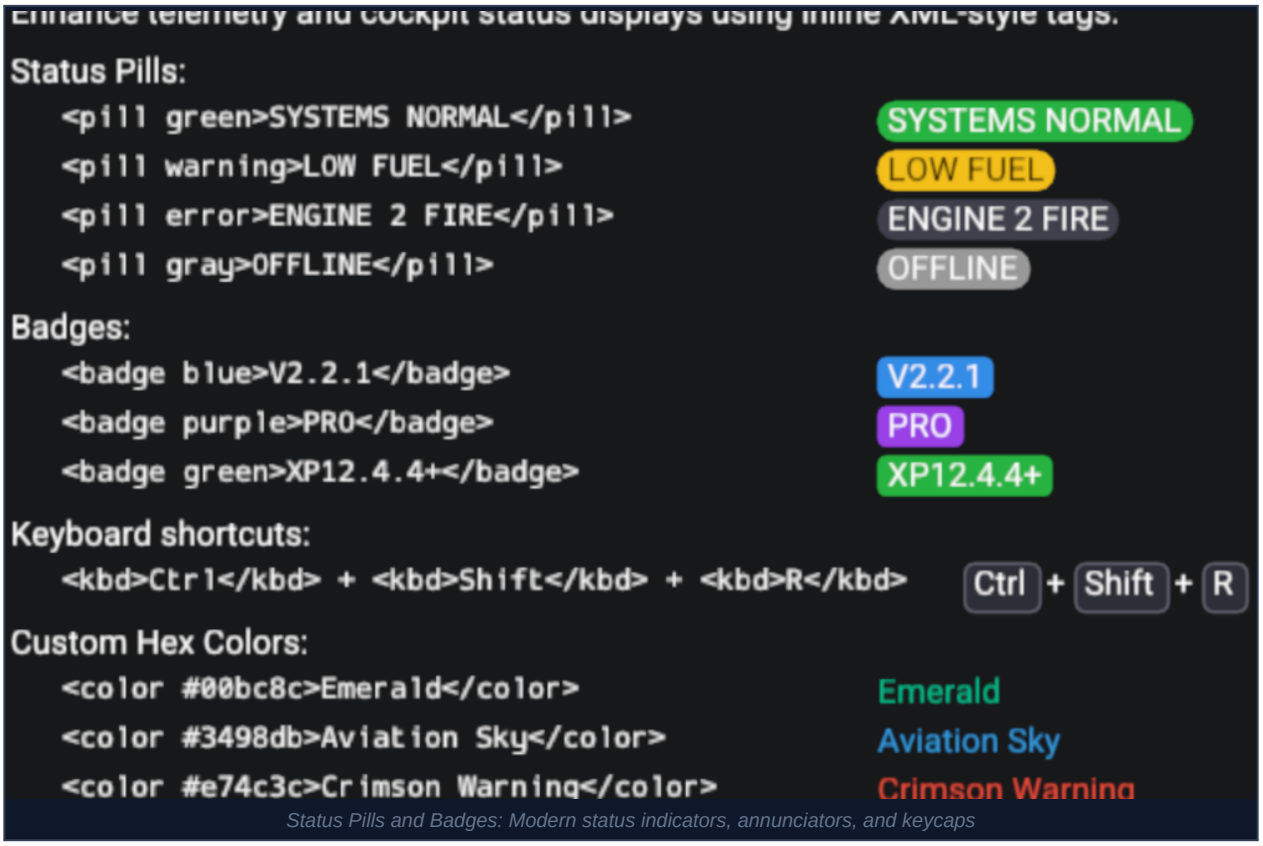
10.4 Badges, Pills & Annunciators (`imgui_badges.h`)

Avionics interfaces require prominent visual annunciations for modes, cautions, and operational states:

```cpp
// 1. Standard Status Pill
ImGui::Pill("ONLINE", ImVec4(0.1f, 0.8f, 0.3f, 1.0f));

// 2. Unary Mode Annunciator with auto-dimming
static bool s_AP_Master = true;
ImGui::AnnunciatorPill("AP ENG", s_AP_Master, ImVec4(0.2f, 0.6f, 1.0f, 1.0f));

// 3. Keyboard Shortcut Keycap
ImGui::Text("AP Disconnect:");
ImGui::SameLine();
ImGui::Kbd("Z");
```

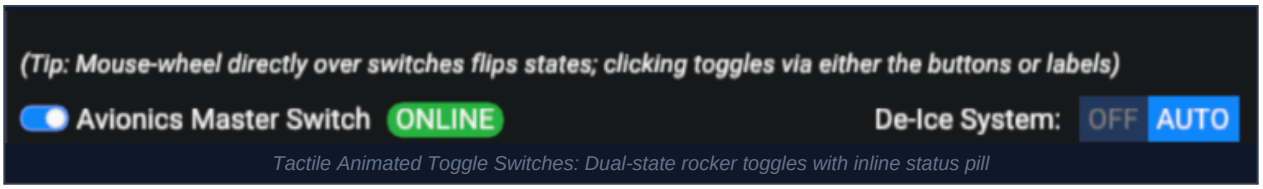


Status Pills and Badges: Modern status indicators, annunciators, and keycaps

10.5 Tactile Animated Toggle Switches (`imgui_toggle_buttons.h`)

Replaces desktop form checkboxes with physical cockpit toggles featuring smooth thumb slide animations:

```cpp
static bool s_StrobeLights = true;
if (ImGui::ToggleButton("Strobe Lights", &s_StrobeLights)) {
    UpdateElectricalBus();
}
```



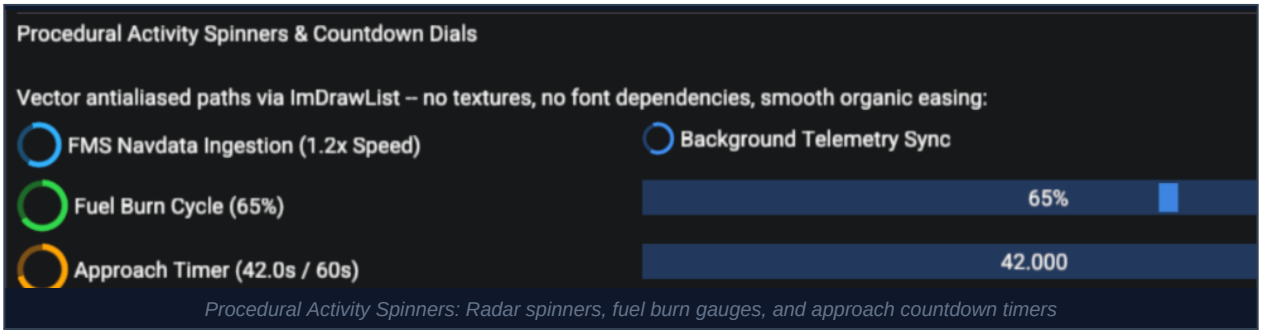
Tactile Animated Toggle Switches: Dual-state rocker toggles with inline status pill

10.6 Procedural Activity Spinners & Countdown Dials (`imgui_spinners.h`)

Procedural radar sweeps and radial timers for approach fixes:

```cpp
// 1. Radar Spinner (Indicates active datalink connection)
ImGui::Spinner("##RadarSweep", 12.0f, 2.5f, IM_COL32(59, 130, 246, 255));

// 2. Radial Timer (Approach countdown)
static float s_LegFraction = 0.65f;
ImGui::RadialTimer("Approach Leg", s_LegFraction, 14.0f, 3.0f, IM_COL32(234, 179, 8, 255));
```



Procedural Activity Spinners: Radar spinners, fuel burn gauges, and approach countdown timers

10.7 Directional Popups, Item Anchoring & SelectPulldown (`imgui_popup_menus.h`)

Provides precision item-anchored popup placement, upward blossoming for bottom-docked status bars, and compact 1-line dataref selectors with Lens UI mouse-wheel interaction:

- **Item-Anchored Popups (`OpenPopupAtItem` & `BeginAnchoredPopup`)**:

Positions dropdowns relative to the preceding item's bounding box (`GetItemRectMin`/`Max`). Supports `ImGuiPopupAnchor_BelowItem` (standard dropdown) or `ImGuiPopupAnchor_AboveItem` (blossoms upward for bottom bars and docks) with font-scale invariance and muscle-memory toggle protection.

- **1-Line Dataref Selector (`SelectPulldown` / `SelectPulldownMD`)**:
- **Scroll Pass-Through When Closed**: Ignores mouse-wheel events while closed so parent scrollable windows never accidentally change settings during scroll navigation.
- **"Lens UI" Selection-Latched Tooltips**: When spinning the mouse wheel inside an open pulldown menu, per-item tooltips dynamically latch onto the newly selected item (`*current_item`) instead of remaining frozen over whichever row the stationary cursor rests on. Deliberate mouse movement (`> 6 px`) cleanly breaks the latch and reverts to standard spatial hover.
- **Predictable Wheel Stepping & Circular Wrapping**: Supports optional `bool wrap = false` and `bool natural_list_order = true` parameters. When `wrap = true`, spinning past boundaries wraps smoothly; `natural_list_order = true` increments down the list on downward wheel rotation (`clicks < 0.0f`).
- **Automatic Focus & Scroll-Into-View**: Navigated items receive `SetItemDefaultFocus()` and scroll into view (`SetScrollHereY()`) in tall menus.
- **Dynamic Cursor Cues**: Hand cursor (`ImGuiMouseCursor_Hand`) on trigger button, resize cursor (`ImGuiMouseCursor_ResizeNS`) over open options.

- **Rich Markdown Tooltip Aliases**: Call `ImGui::SelectPulldownMD(...)` or `ImGui::MD::SelectPulldown(...)` with rich Markdown tooltips (`TimedTooltip::MD::TextProxy`).

```cpp
// 1. Standard Downward Pulldown with Lens UI tooltips and circular wrap:
static int s_SelectedCam = 1;
const char* const kCamModes[] = { "Circle", "Spot", "Tower", "Cockpit" };
const char* const kCamTips[]  = { "**Circle**\nFollow aircraft", "**Spot**\nFixed distance chase", "**Tower**\n
if (ImGui::SelectPulldownMD("##CameraMode", &s_SelectedCam, kCamModes, 4, kCamTips, 130.0f, ImGuiPopupAnchor_Bel
    UpdateCamera(s_SelectedCam);
}

// 2. Upward Blossoming Pulldown (Status Bars & Bottom Docks):
static int s_PowerMode = 0;
const char* const kPowerModes[] = { "Battery", "APU", "External Power" };
if (ImGui::SelectPulldown("##PowerMode", &s_PowerMode, kPowerModes, 3, nullptr, 140.0f, ImGuiPopupAnchor_AboveIt
    UpdatePowerSource(s_PowerMode);
}
```

Appendix A: Preprocessor Opt-Out Configuration & Build Matrix

For projects with strict binary footprint requirements or codebases that define conflicting symbols, ImGee provides a comprehensive two-tier opt-out matrix. Define any of the following macros *before* including `contrib.h` or `imgee.h`:

A.1 Master Container & Suite Toggles

Preprocessor Define	Scope	Description
`CONTRIB_DISABLE_ALL`	Container	Completely disables the entire `contrib/` folder. Zero contrib code compiled.
`IMGEE_DISABLE_ALL` (or `CONTRIB_IMGEE_DISABLE_ALL`)	Suite	Excludes the entire ImGee suite while leaving other contrib packages active.

A.2 Subsystem & Feature Opt-Outs

Canonical Macro Directive	Legacy Alias	Description
`IMGEE_DISABLE_EXTENSIONS`	`CONTRIB_DISABLE_EXTENSIONS`	Excludes Typography, Markdown, and TimedTooltip.
`IMGEE_DISABLE_WIDGETS`	`CONTRIB_DISABLE_WIDGETS`	Excludes all rotary knobs, sliders, badges, toggles, and popups.
`IMGEE_DISABLE_TIMED_TOOLTIP`	`CONTRIB_DISABLE_TIMED_TOOLTIP`	Excludes stationary delayed tooltip engine.
`IMGEE_DISABLE_MARKDOWN`	`CONTRIB_DISABLE_MARKDOWN`	Excludes Markdown parser, GFM tables, and callouts.
`IMGEE_DISABLE_TYPOGRAPHY`	`CONTRIB_DISABLE_TYPOGRAPHY`	Excludes `ImFontRole` and `ImGui::StyledText`.
`IMGEE_DISABLE_ICONS`	`CONTRIB_DISABLE_ICONS`	Excludes procedural vector icon rendering engine.
`IMGEE_DISABLE_ICON_BUTTON`	`CONTRIB_DISABLE_ICON_BUTTON`	Excludes `IconButton` and `ButtonIcon` push buttons.
`IMGEE_DISABLE_KNOBS`	`CONTRIB_DISABLE_KNOBS`	Excludes avionics rotary knobs, dials, and detents.
`IMGEE_DISABLE_MOUSE_WHEEL_CONTROLS`	`CONTRIB_DISABLE_MOUSE_WHEEL_CONTROLS`	Excludes wheel sliders, drags, combos, and reset buttons.
`IMGEE_DISABLE_BADGES`	`CONTRIB_DISABLE_BADGES`	Excludes status badges, pills, annunciators, and keycaps.

Canonical Macro Directive	Legacy Alias	Description
`IMGEE_DISABLE_POPUP_MENUS`	`CONTRIB_DISABLE_POPUP_MENUS`	Excludes directional anchored popups (`AboveItem`) and pulldowns.
`IMGEE_DISABLE_TOGGLE_BUTTONS`	`CONTRIB_DISABLE_TOGGLE_BUTTONS`	Excludes animated physical toggle switches.
`IMGEE_DISABLE_CLICKABLE_LINKS`	`CONTRIB_DISABLE_CLICKABLE_LINKS`	Excludes OS hand cursor links (`ButtonLink`, `TextLink`).
`IMGEE_DISABLE_SPINNERS`	`CONTRIB_DISABLE_SPINNERS`	Excludes activity spinners and countdown timer dials.
`IMGEE_ENABLE_GLOBAL_NAMESPACES`	`CONTRIB_ENABLE_GLOBAL_NAMESPACES`	Publishes `Markdown::`, `MD::`, and `TimedTooltip::` to global scope.

NOTE

Legacy Alias Bridge: For projects written against earlier versions of ImgWindow, legacy switches like `CONTRIB_DISABLE_WIDGETS`, `CONTRIB_DISABLE_BADGES`, and `CONTRIB_DISABLE_MARKDOWN` are automatically mapped to their canonical `IMGEE_*` counterparts with zero build warnings.

Document Version: 1.0.0 • Author: Steven L. Goldberg (@slgoldberg) and the Rogers • ImGee Suite

ImGee Extensions Suite

This directory contains standalone, feature-complete **Extensions** for Dear ImGui within the **ImGee** suite.

What is an Extension?

In ImGee, an **Extension** is a self-contained subsystem providing substantial functionality beyond standard UI widgets. Extensions typically:

- Maintain internal or persistent state between frames.
- Require custom configuration structs or setup steps.
- Encapsulate complex algorithms (e.g., text parsing, layout pre-calculation, delayed timers).
- May integrate or adapt third-party libraries.

Namespacing Philosophy: Omar Cornut's Standard & Sub-Namespaces

A common question from newcomers is: *"Why not invent a separate root namespace like* **`extgui::`**, **`eximgui::`**, *or* **`extimgui::`** *to distinguish extension code?"*

The answer lies in **Dear ImGui's official architectural philosophy**:

1. Omar Cornut's Ecosystem Standard (Single Point of Discovery)

In official Dear ImGui extensions and ecosystem projects (such as **`implot`**, **`imgui_club`**, and **`imnodes`**), Omar Cornut explicitly advocates extending **`namespace ImGui { ... }`**.

- **Autocomplete & Discoverability:** Modern C++ developers rely heavily on IDE autocomplete. When a developer types **`ImGui::`**, they expect to discover every capability available within their GUI framework in one place.
- **Zero Cognitive Overhead:** Creating detached root namespaces (**`extgui::`**) forces developers to remember which feature lives in which foreign namespace, requires scattered **`using namespace`** declarations, and creates awkward syntax when passing native ImGui types across boundaries.

2. The Power of Second-Level Sub-Namespaces

While extending **`namespace ImGui`** is essential, dumping heavy packages directly into the root namespace risks colliding with current or future Dear ImGui core functions.

The elegant solution is **second-level sub-namespaces** (**`ImGui::<PackageName>::`**):

```[CPP]
namespace ImGui {
    namespace <PackageName> {
        // Functions mirror familiar ImGui patterns
    }
}
```

- **Zero Root Pollution:** The root **`ImGui::`** namespace stays clean, pristine, and safe from symbol collisions.
- **Mirroring Native ImGui Vocabulary:** Sub-namespaces allow us to reuse Dear ImGui's standard vocabulary (**`Text`**, **`TextWrapped`**, **`CalcTextSize`**):

```[CPP]
  ImGui::Markdown::TextWrapped("Configure your **settings** below.");
  ImGui::TimedTooltip::Text("Hover-delayed stationary tooltip.");
```

TIP

Forgiving Aliases ("It Just Works"): Prefer shorter names or don't want to think about namespaces? * `ImGui::MD::` is an official, zero-overhead shorthand alias for `ImGui::Markdown::`. * Root shorthands like `ImGui::TextMD(...)` and cross-extension bridges like `ImGui::TimedTooltip::MD::Text` work seamlessly. * Want to skip `ImGui::` entirely? `#define CONTRIB_ENABLE_GLOBAL_NAMESPACES` publishes `Markdown::`, `MD::`, and `TimedTooltip::` directly to the global scope.

Current Extensions

Extension	Directory	Canonical Namespace	Shorthand Alias	Description
Markdown	`Markdown/`	`ImGui::Markdown::`	`ImGui::MD::`	Markdown parser and renderer based on `enkisoftware/imgui_markdown`, enhanced with dynamic font scaling, token lookahead, true hanging indents, rich parameterized styling tags (`<color>`, `<backdrop>`, `<badge>`), procedural vector icons (`<icon ...>`), and native vector button faces (`ButtonMD`).
TimedTooltip	`TimedTooltip/`	`ImGui::TimedTooltip::`	*(Root & Bridge)*	Advanced stationary and delayed tooltip engine with automatic bounding box pre-calculation, viewport edge-clamping, and support for both plain-text and Markdown tooltips.
Typography	`Typography/`	`ImGui::Typography::`	`ImGui::` *(Root)*	Semantic font role hierarchy (`ImFontRole`), commutative scoped font guards (`ScopedFontRole`), and the inline `StyledText` engine with continuous baseline alignment for badges, pills, and textures.

Contribution Guidelines for Extensions

When creating or porting a new extension:

1. **Self-Contained Folder:** Place all headers, implementation files, and assets in a dedicated folder under `contrib/extensions/<ExtensionName>/`.
2. **Single Header Entrypoint:** Provide a primary header (e.g. `<extension_name>_ext.h` or `<extension_name>.h`) that consumers can include with one `#include` directive.
3. **No Mandatory Global State:** Avoid requiring global mutable state where possible. If state is needed, manage it within an explicit context struct or provide clean default fallbacks.
4. **ImGui Version Compatibility:** Gate version-specific ImGui calls using `#if IMGUI_VERSION_NUM >= ...` so the extension builds cleanly across supported Dear ImGui versions.
5. **Documentation:** Every extension folder must include a comprehensive `README.md` containing:

• Architecture and features.

• Minimal copy-pasteable usage example.

- Configuration options and callbacks.
- Any licensing or upstream credit.

TimedTooltip Extension

The **TimedTooltip** extension provides an advanced, stationary tooltip engine designed specifically for Dear ImGui within X-Plane plugins.

Unlike standard Dear ImGui tooltips which move with the mouse and can flicker, obstruct UI elements, or run off-screen, `TimedTooltip` locks in place, calculates dynamic viewport boundaries, preserves custom interactive cursors, isolates its styling hermetically from parent windows, and supports both plain-text and rich Markdown content.

> **Author / Maintainer:** Steven L. Goldberg (@slgoldberg)

> **License:** BSD 3-Clause (see ImgWindow LICENSE)

Key Features

- **Configurable Delay & Lifetime:**
- **Low Watermark (Hover Delay):** Prevents tooltip spam by waiting for the user to pause over a widget before showing.
- **Active Input Low-Water Reset:** Spinning the mouse wheel (`io.MouseWheel` / `io.MouseWheelH`) or dragging (`IsMouseDragging`) over any widget immediately slams the hover timer back to 0 and dismisses any active tooltip, preventing tooltips from popping up or obstructing adjustments during continuous wheel scrolling or slider interaction.
- **High Watermark (Auto-Dismiss):** Automatically hides tooltips after a readable duration, with adaptive time-scaling for taller/multi-line text.
- **Grace Period:** Forgives micro-slips off the target widget so users don't lose their place.
- **Wiggle Latch:** Micro-mouse movements reset the auto-hide timer so users can continue reading long descriptions.
- **Hermetic Style Isolation:**
- Pushes and pops its own `WindowPadding`, `ItemSpacing`, `FramePadding`, and `IndentSpacing`. Outer parent window style modifications (such as `ItemSpacing.x = 0` in menu bars or compressed button pads) can never distort tooltip typography, and tooltip styles never leak back out.
- Precise vertical line pitch (`item_spacing.y = 0.0f`) preserves the exact 15px native font leading in X-Plane, preventing vertical bloat and screen overrun.
- **Smart Bounds Clamping & Docking:**
- **Cursor-Safe Viewport Clamping (Flip Above Cursor):** If a tooltip would clip off the bottom of the screen (e.g. over a bottom control strip or footer slider), it automatically flips **above the mouse cursor**, ensuring the tooltip never covers the cursor or the widget being manipulated.
- **Leftward Shift:** If a tooltip approaches the right screen border, it automatically shifts leftwards.
- **Adaptive Wrap:** Shrink-wraps tightly around concise text while wrapping paragraphs at a readable width (80% of available viewport width by default).
- **Interactive Cursor Protection & Click-to-Toggle Ergonomics:**
- Preserves and restores custom widget cursors (e.g., `ImGuiMouseCursor_Hand` or `ImGuiMouseCursor_ResizeEW`), and automatically shows a Hand cursor on passive text/labels that have a tooltip attached.
- Clicking passive text, right-side Slider/Drag/Combo labels, dimmed at-default reset buttons, or standalone `*Link` widgets immediately toggles the tooltip ON/OFF without waiting for `hover_delay_cycles`.
- **Multi-`ImGuiContext` & Overlapping-Widget Safety:**
- Isolates tooltip state and wheel-selection latches per `ImGuiContext*` so multiple `ImgWindow` instances rendering in the same X-Plane draw cycle never interfere with one another.
- **Foreground-Widget Hover Priority:** When a button sits on the same line as a full-width background `Selectable(..., ImGuiSelectableFlags_SpanAllColumns)`, `TimedTooltip` automatically gives tooltip priority to the earlier foreground widget on the row without requiring manual hover-suppression flags.

- **Plain Text & Markdown Support (Order-Independent):**
- Native auto-wrapping plain-text via `Text(...)`, single-line non-wrapping plain-text via `TextUnformatted(...)`, rich auto-wrapping Markdown via `TextMD(...)`, and single-line non-wrapping Markdown via `TextUnformattedMD(...)`.
- `imgui_tooltip_ext.h` and `imgui_markdown_ext.h` can be `#include`d in **either order** and the `TextMD` / `TextUnformattedMD` bridge is synthesized automatically.

Quick Start

1. Include the Header

```
[CPP]
#include "contrib/extensions/TimedTooltip/imgui_tooltip_ext.h"
```

2. Basic Usage (Plain Text)

Attach a timed tooltip directly after drawing any ImGui widget:

```
[CPP]
if (ImGui::Button("Save Flight Plan")) {
    SavePlan();
}
// Automatically attaches to the preceding widget:
ImGui::TimedTooltip::Text("Saves the active route to disk as an .fms file.");
```

3. Markdown Tooltips

If using the companion **Markdown** extension, simply call `TextMD`:

```
[CPP]
#include "contrib.h"

ImGui::Button("Autopilot Nav");
ImGui::TimedTooltip::TextMD("Arms **LNAV** mode. *Requires active GPS waypoint.*");
```

> **TIP**
>
> **Forgiving Aliases:** Prefer explicit sub-namespaces? `ImGui::TimedTooltip::Markdown::Text(...)` and `ImGui::TimedTooltip::MD::Text(...)` work too! If you prefer flipping the order (`ImGui::Markdown::TimedTooltip::Text`), that works identically. Clickable links inside tooltips are supported via `TextMDLink(...)`.

Configuration (`ImGui::TimedTooltip::Config`)

The `TooltipConfig` (aliased as `ImGui::TimedTooltip::Config`) controls timing, dimensions, typography, and colors.

Configuration Fields Reference

Field	Type	Default	Description
`hover_delay_cycles`	`int`	`65`	Frame cycles the mouse must hover before the tooltip appears (~1.0s at 60fps).

Field	Type	Default	Description
`max_display_cycles`	`int`	`504`	Frame cycles before the tooltip auto-dismisses. Set negative to disable auto-hide.
`grace_period_cycles`	`int`	`12`	Allowable frame cycles off the widget before cancelling hover state.
`size_skew_multiplier`	`float`	`0.5f`	Extra cycles added to display timeout per pixel of vertical height.
`enable_wiggle_latch`	`bool`	`true`	When `true`, subtle mouse movements inside the tooltip reset the dismiss timer.
`default_wrap_width`	`float`	`360.0f`	Target comfortable sticky-note width (~50-60 chars). Short text shrink-wraps tightly to its natural size.
`min_wrap_width`	`float`	`240.0f`	Minimum width boundary before forced truncation/overflow.
`max_wrap_width`	`float`	`600.0f`	Maximum width ceiling for wide documentation/manuals.
`padding`	`ImVec2`	`(8.0f, 6.0f)`	Internal window padding around the tooltip contents.
`item_spacing`	`ImVec2`	`(4.0f, 0.0f)`	Item spacing between lines. `y = 0.0f` matches native font leading.
`frame_padding`	`ImVec2`	`(2.0f, 1.0f)`	Internal padding for frame elements inside the tooltip.
`indent_spacing`	`float`	`20.0f`	Standard indent spacing between nested list levels.
`bullet_spacing`	`float`	`4.0f`	**[Markdown Tooltips Only]** Gap in pixels between bullet glyph and following text when rendering rich lists via `TextMD`. (Ignored for plain-text tooltips.)
`font_scale`	`float`	`0.0f`	Window font scale multiplier (`0.0f` = automatically inherit the ambient font scale from the calling parent window; `>0.0f` = explicit scale override).
`bg_color`	`ImVec4`	Solid yellow	Window and popup background color (`#FFF299`, 100% opaque `alpha = 1.0f`).
`border_color`	`ImVec4`	Dark yellow	Tooltip border outline color (`#CCBF66`).
`text_color`	`ImVec4`	Jet black	Tooltip text and separator color (`#000000`).

NOTE

Resolution-Independent Base Dimensions: All dimensional settings (`default_wrap_width`, `min_wrap_width`, `max_wrap_width`, `padding`, `item_spacing`, `frame_padding`, and `indent_spacing`) are defined in **base design boxels** (at 1.0× baseline). The tooltip engine resolves the active ambient scale (`ImGui::GetFontSize() / 14.0f` or explicit `font_scale`) and automatically scales constraints and padding dynamically. Callers do not need to manually multiply configuration values by font zoom factors.

Usage Patterns: Global Setup vs. Per-Call Overrides

Pattern A: Global Setup (Once at Plugin Startup)

For 95% of use cases, configure your desired tooltip personality once during plugin startup or window creation:

```cpp
// In your plugin initialization:
static ImGui::TimedTooltip::Config s_MyTooltipConfig;
s_MyTooltipConfig.hover_delay_cycles = 45;                          // Snappier hover delay
s_MyTooltipConfig.padding            = ImVec2(8.0f, 6.0f);          // Compact padding
s_MyTooltipConfig.item_spacing       = ImVec2(4.0f, 0.0f);          // Tight line pitch
s_MyTooltipConfig.bg_color           = ImVec4(1.0f, 0.95f, 0.6f, 1.0f); // 100% opaque sticky note

ImGui::TimedTooltip::SetDefaultTooltipConfig(&s_MyTooltipConfig);
```

Every subsequent call to **`ImGui::TimedTooltip::Text(...)`** and **`ImGui::TimedTooltip::TextMD(...)`** will automatically use this configuration without needing any per-call arguments.

Pattern B: Per-Call Override (Instant & Dynamic)

If a specific widget needs unique styling—for example, a wider maximum width, distinct background color, or custom padding—you can fetch a copy of the active configuration using **`ImGui::TimedTooltip::GetConfig()`**, tweak the desired fields, and pass it directly to **`Text`** or **`TextMD`**:

```cpp
// Safely copies the active global config (or built-in defaults if no global config was set):
ImGui::TimedTooltip::Config wideConfig = ImGui::TimedTooltip::GetConfig();
wideConfig.max_wrap_width = 800.0f;                       // Allow wider manual formatting
wideConfig.bg_color       = ImVec4(0.15f, 0.15f, 0.20f, 0.95f); // Dark HUD styling
wideConfig.text_color     = ImVec4(1.0f, 1.0f, 1.0f, 1.0f);

ImGui::Button("System Diagnostics");
// Pass custom configuration directly:
ImGui::TimedTooltip::TextMD(&wideConfig, "### System Status\n* Bus 1: **Normal**\n* Bus 2: **Normal**");
```

Pattern C: Low-Level Custom Layout Blocks

For completely custom tooltip content containing arbitrary ImGui widgets, use **`BeginStationaryTooltip`** and **`EndStationaryTooltip`**:

```cpp
ImGui::Button("Engine Monitor");
if (ImGui::TimedTooltip::BeginStationaryTooltip("EngineMonitorTooltip")) {
    ImGui::Text("EGT: 640 C");
    ImGui::ProgressBar(0.75f, ImVec2(150, 15));
    ImGui::TimedTooltip::EndStationaryTooltip("EngineMonitorTooltip");
}
```

Hermetic Style Isolation

In Dear ImGui, child windows and popups can inadvertently inherit active layout styles from their parent windows. If an outer menu bar or custom toolbar has set **`ItemSpacing.x = 0.0f`** or compressed **`FramePadding`** to pack buttons tightly, standard tooltips would inherit those distorted spacing values.

`TimedTooltip` isolates itself completely:

1. When opening a tooltip proxy, **`BeginStationaryTooltipProxy`** pushes:

- **`ImGuiStyleVar_WindowPadding`** (**`config->padding`**)

- `ImGuiStyleVar_ItemSpacing` (`config->item_spacing`)
- `ImGuiStyleVar_FramePadding` (`config->frame_padding`)
- `ImGuiStyleVar_IndentSpacing` (`config->indent_spacing`)

2. When closing, `EndStationaryTooltip` pops all four style variables in exact reverse order.

This guarantees that:

- **No outer window hacks leak inward:** Tooltips always render with crisp, predictable typography regardless of what the calling window modified.
- **No tooltip styles leak outward:** Calling windows retain their exact style state without needing defensive push/pop blocks.

Screen Clamping & Docking

Tooltips automatically check the active display dimensions (`ImGui::GetIO().DisplaySize`):

- **Bottom Edge Protection:** If `render_pos.y + height` exceeds the screen floor, the anchor position automatically shifts upward so the entire tooltip remains visible on screen.
- **Right Edge Protection:** If `render_pos.x + width` exceeds the screen edge, the tooltip shifts leftwards to keep all content in view.

Click Forwarding & Automatic Dismissal

Tooltips attached to interactive or proxy-instrumented widgets (like `ButtonLink`, chevron menus, or `TextProxy` / `TextMDProxy` items) automatically intercept clicks via `ImGui::IsItemClicked()`:

- **Immediate Dismissal:** Clicking an active widget immediately dismisses the active tooltip, preventing tooltips from lingering over open menus or blocking post-click interactions.
- **Toggle on Idle Click:** Clicking an idle/non-actionable widget (such as a dimmed reset button or standalone info link) cleanly toggles the explanation tooltip on/off.

X-Plane 12 Panel Graphics (Linear Blending & Font Scaling)

1. 100% Solid Opacity (`alpha = 1.0f`)

In legacy OpenGL, plugins composited in uncorrected gamma space, where semi-transparent yellow cards (`alpha = 0.95f`) appeared largely solid.

In X-Plane 12's modern **Panel Graphics** pipeline (Metal on macOS, Vulkan on Windows/Linux), compositing is computed in **physically linear color space**. Even a minor 5% transparency allows intense runway lights, aircraft labels, and bright background elements to illuminate the pixels under dark text. Furthermore, linear antialiasing blends 50% edge coverage at $\approx 73\%$ background luminance ($(0.5)^{1/2.2} \approx 0.73$), optically eroding dark letter stems.

To ensure crisp, dark typography without background wash-out, `TooltipConfig::bg_color` defaults to `1.0f` **solid opacity** (`ImVec4(1.0f, 0.95f, 0.6f, 1.0f)`).

2. Automatic Ambient Font Scale Inheritance

When `TooltipConfig::font_scale` is left at `0.0f` (the default), the tooltip engine automatically queries and inherits the ambient window font scale of the calling widget:

- **Custom Window Zoom:** If a child window or floating palette is zoomed to 124%, hovering a widget inside that window renders the tooltip scaled at that exact same 124% factor.

- **Accessibility / Senior Citizen Mode:** If a global or per-window 17pt font mode is active, the tooltip inherits the enlarged scale seamlessly.
- **Pre-Calculation Accuracy:** The 2-pass size pre-calculator (`MD::CalcMarkdownSize`) computes wrap boundaries and vertical heights using this exact inherited scale factor, guaranteeing zero text clipping and zero scrollbar flutter.

API Summary

```cpp
namespace ImGui {
    namespace TimedTooltip {
        // High-level Plain Text (auto-wraps to TooltipConfig bounds; returns true when displayed):
        bool Text(const char* fmt, ...);
        bool Text(const TooltipConfig* config, const char* fmt, ...);
        bool Text(const std::string& text, const TooltipConfig* config = nullptr);

        // Single-line / Non-Wrapping Plain Text (mirrors ImGui::TextUnformatted):
        bool TextUnformatted(const char* text, const char* text_end = nullptr, const TooltipConfig* config = nullptr);
        bool TextUnformatted(const std::string& text, const TooltipConfig* config = nullptr);

        // High-level Plain Text Proxy (explicit ID and hover state):
        bool TextProxy(ImGuiID id, bool is_hovered, const char* fmt, ...);
        bool TextProxy(ImGuiID id, bool is_hovered, const TooltipConfig* config, const char* fmt, ...);
        bool TextProxy(ImGuiID id, bool is_hovered, const std::string& text, const TooltipConfig* config = nullptr);
        bool TextProxy(const char* str_id, bool is_hovered, const char* fmt, ...);
        bool TextProxy(const char* str_id, bool is_hovered, const std::string& text, const TooltipConfig* config = nullptr);
        bool TextUnformattedProxy(ImGuiID id, bool is_hovered, const char* text, const char* text_end = nullptr, const TooltipConfig* config = nullptr);

        // High-level Markdown (requires imgui_markdown_ext.h in either include order; returns true when displayed):
        bool TextMD(const char* fmt, ...);
        bool TextMD(const TooltipConfig* config, const char* fmt, ...);
        bool TextMD(const std::string& markdown_text, const TooltipConfig* config = nullptr);

        // Single-line / Non-Wrapping Markdown (never wraps unless \n or PushTextWrapPos is active):
        bool TextUnformattedMD(const char* text, const char* text_end = nullptr, const TooltipConfig* config = nullptr);
        bool TextUnformattedMD(const std::string& markdown_text, const TooltipConfig* config = nullptr);

        // High-level Markdown Proxy (explicit ID and hover state):
        bool TextMDProxy(ImGuiID id, bool is_hovered, const char* fmt, ...);
        bool TextMDProxy(ImGuiID id, bool is_hovered, const TooltipConfig* config, const char* fmt, ...);
        bool TextMDProxy(const char* str_id, bool is_hovered, const char* fmt, ...);
        bool TextMDProxy(ImGuiID id, bool is_hovered, const std::string& markdown_text, const TooltipConfig* config = nullptr);
        bool TextMDProxy(const char* str_id, bool is_hovered, const std::string& markdown_text, const TooltipConfig* config = nullptr);
        bool TextUnformattedMDProxy(ImGuiID id, bool is_hovered, const char* text, const char* text_end = nullptr, const TooltipConfig* config = nullptr);

        // Explicitly opt the preceding widget into click-to-toggle tooltip behavior for the current frame:
        void ToggleOnClick(bool enable = true);

        // Configuration:
        using Config = TooltipConfig;
        using State  = TooltipState;
        void SetDefaultTooltipConfig(const Config* config);
        const Config*& GetDefaultTooltipConfig();
        const Config& GetCurrentTooltipConfig();
        Config GetConfig();

        // Low-level lifecycle controls:
        bool BeginStationaryTooltipProxy(ImGuiID id, bool is_hovered, const Config* cfg = nullptr);
        bool BeginStationaryTooltip(ImGuiID id, const Config* cfg = nullptr);
        bool BeginStationaryTooltip(const char* str_id, const Config* cfg = nullptr);
        void EndStationaryTooltip(ImGuiID id);
        void EndStationaryTooltip(const char* str_id);
        void ClearActiveTooltip();
    }
}
```

Markdown Extension

The **Markdown** extension provides rich Markdown parsing and rendering for Dear ImGui within ImgWindow, based on `enkisoftware/imgui_markdown` and heavily enhanced for X-Plane plugin user interfaces.

It allows you to embed rich text formatting, headers, links, lists, and custom interactive widgets directly into your ImGui windows, settings panels, and stationary tooltips.

> **Author / Maintainer:** Steven L. Goldberg (@slgoldberg)

> **Original Upstream:** Juliette Foucaut (@juliettef) & Doug Binks (@dougbinks) — Repository: enkisoftware/imgui_markdown

> **License:** BSD 3-Clause (Extension Wrapper; see ImgWindow LICENSE) & zlib (Upstream Parser)

Architectural Overview: Foundation vs. Extension Wrapper

The Markdown system is structured as a clean two-layer architecture:

1. **Base Engine (`imgui_markdown.h`):**

- Upstream parser originally created by Doug Binks and Juliette Foucaut (`enkisoftware/imgui_markdown`).
- Responsible for raw character scanning, CommonMark tokenization, AST-like block dispatch, and low-level C-style callbacks (`formatCallback`, `linkCallback`, `imageCallback`, `tooltipCallback`).
- Our fork incorporates crucial low-level fixes for CommonMark punctuation lookahead and bullet list token isolation.

2. **ImgWindow Extension Wrapper (`imgui_markdown_ext.h`):**

- High-level, ergonomic C++ layer designed specifically for X-Plane plugins and modern ImGui applications.
- Encapsulates all functionality inside the second-level `ImGui::Markdown::` namespace, with `ImGui::MD::` provided as an official, zero-overhead namespace alias (`namespace ImGui { namespace MD = Markdown; }`).
- Eliminates the upstream symbol collision: Upstream's raw C-style function `ImGui::Markdown(text, len, config)` has been moved to `ImGui::Markdown::Render(...)` (and `ImGui::MD::Render(...)`). This prevents beginners from accidentally invoking the raw unconfigured parser and frees `ImGui::Markdown` to be a clean, first-class C++ namespace.
- Introduces an XML/HTML-style inline tag pre-processor (`<tag>`, `<color>`, `<backdrop>`, `<badge>`, `<btn>`) with custom tag callback registration via `ImGui::Markdown::RegisterTag`.
- Solves Dear ImGui layout hurdles: typographic line-wrap lookahead (eliminating mid-word breaks across styles and links), mathematical hanging indents for wrapped multi-line bullets, and ambient font scale inheritance (`ImGui::GetFontScale()`).
- Provides non-rendering layout measurement via `ImGui::Markdown::CalcTextSize` (defeating ImGui viewport culling to measure exact dimensions for floating windows and stationary tooltips).
- Directly interfaces with the companion `TimedTooltip` extension via bidirectional bridges (`ImGui::TimedTooltip::Markdown::Text` / `ImGui::TimedTooltip::MD::Text` and `ImGui::Markdown::TimedTooltip::Text` / `ImGui::MD::TimedTooltip::Text`).

Architectural Enhancements over Upstream

- **Collision-Free Canonical Namespace (`ImGui::Markdown::` & `ImGui::MD::`):**
- Primary namespace is `ImGui::Markdown::Text(...)` for full clarity.
- Shorthand alias `ImGui::MD::Text(...)` is 100% interchangeable.
- Root convenience aliases `ImGui::TextMD(...)` / `ImGui::TextMarkdown(...)` allow drop-in usage matching `ImGui::Text(...)`.

- Opt-in global convenience (`#define CONTRIB_ENABLE_GLOBAL_NAMESPACES` or `#include "contrib/contrib-global.h"`) publishes `Markdown::Text(...)` and `MD::Text(...)` to the global scope for developers seeking minimum boilerplate.
- **CommonMark Backslash Escaping:**
- Escapes any ASCII punctuation character (`\*`, `\#`, `\_`, `\<`, `\>`, `\[`, `\]`, `\(`, `\)`, `\\`, etc.) to render literally without triggering emphasis, headers, HTML tags, or links.
- Ensures clean display of technical expressions like `C\*` or `[X-Plane 12]\(build\)`.
- **Strikethrough & Underline Typography:**
- **Strikethrough:** Full support for GitHub Flavored Markdown `~~deleted text~~` and legacy ABC `$$struck text$$`, as well as HTML tags (`<s>`, `<del>`, `<strike>`).
- **Underline:** Supports CriticMarkup / Markdown Extended `++underlined text++` and HTML tags (`<u>`, `<ins>`) out-of-the-box. Setting `mdConfig.underlineWithUnderscores = true` allows Discord/ABC-style `__underlined text__` to render as underline instead of bold.
- **Pixel-Aligned Rules:** Centered midline for strikethrough and baseline-aligned underline scale cleanly across all display DPIs and custom font sizes.
- **Ordered & Unordered Lists with True Hanging Indents:**
- **Ordered Lists:** Automatically recognizes numbered items (`1.` , `2.` , etc.) and formats the number prefix with a clean hanging indent matching unordered bullets.
- **Unordered Lists:** Replaces cramped upstream bullet spacing with scalable, comfortable breathing room (`bulletSpacing` defaulting to a clean 4px).
- Mathematically maintains true hanging indents across wrapped multi-line list items and continuation lines, keeping all wrapped text cleanly aligned to the text column rather than under the bullet glyph or number.
- Preserves leading space depth across emphasis chunks and links so nested sub-bullets never pop out of alignment.
- Broadens list marker syntax to standard Markdown (`*`, `-`, `+`) with or without leading whitespace, with strict token isolation so bullet glyphs never hijack inline styling into rogue italic runs.
- **Nested Indent Balance:** Accurately unindents when nested lists terminate, preventing indentation from leaking into subsequent paragraphs or blocks.
- **Thematic Breaks / Horizontal Rules (CommonMark 4.1):**
- Full support for thematic breaks via `---`, `***`, or `___` (with optional internal/trailing whitespace).
- Automatically renders clean visual dividers via `ImGui::Separator()`.
- **Smart Spacing & Void Prevention:** Intelligently suppresses redundant vertical margins when separators immediately precede or follow headings, avoiding cavernous whitespace voids.
- **Fenced Code Blocks with Width Clamping & Continuation Soft-Wrapping:**
- Multi-line code blocks (` ``` and ~~~) clamp their container width to available window content space (`avail_width`) while guaranteeing minimum toolbar control width.
- Continuation lines soft-wrap naturally without horizontal clipping.
- **CommonMark 4.5 Indentation Stripping:** Code blocks nested inside lists or indented sections automatically strip up to N spaces of leading indentation matching the opening fence, preventing code from being pushed awkwardly to the right inside the container.
- **Indented Closing Fences:** Closing fences indented by ≥ 4 spaces or tabs are properly recognized, preventing list continuation text from being accidentally swallowed.
- **Scoped Copy Button:** Integrated vector copy button with checkmark animation and 1.5s transient visual feedback. Scoped via `ImGuiStorage` to ensure multiple code blocks in the same window maintain their feedback independently.
- **Mode Toggle with Mouse-Wheel Scroll-Defeat:** The animated `Raw [===O] Preview` toggle switch automatically defeats mouse-wheel adjustments when inside scrollable parent viewports, preventing accidental mode switches when

scrolling through documentation.

- **Soft-Wrap Leading Space Suppression:**
- Forced line breaks on wrapped prose cleanly suppress carried-forward leading spaces, ensuring continuation lines align flush against the left text margin.
- **Layout Pre-Calculation (`CalcSize` / `CalcTextSize`):**
- Exposes `ImGui::MD::CalcSize(...)` (aliased as `CalcTextSize`) to pre-measure exact wrapped multi-line Markdown dimensions without rendering to the screen—crucial for sizing OS floating windows, child frames, and stationary tooltips.
- Defeats ImGui viewport culling by employing pass-unique offscreen window hashes.
- **Interactive CommonMark Tables with Sorting & Bi-Directional State Serialization:**
- Parses standard CommonMark pipe tables with column alignment (`:---`, `:---:`, `---:`), natural cell content measurement, and rich inline Markdown/tags inside cells.
- **Tristate Interactive Sorting:** Click any header to cycle Ascending ↑ → Descending ↓ → Unsorted, with responsive hand cursor hover and rendered sort indicator arrows.
- **Column Divider Resizing:** Drag any column boundary separator to customize column widths dynamically.
- **Total Width Border Grip:** Drag the right-border resize handle to stretch or contract table width up to 5x available window width (smooth horizontal scrolling in narrow viewports).
- **A/B Double-Click Toggle Memory:** Double-clicking the right border grip toggles back and forth between the baseline preset and user-modified geometry with 100% memory of column widths and total width.
- **Two-Tier Comment Persistence:** Transparently reads and writes Author Baseline Presets (`<!-- table-preset: ... -->`) and User State (`<!-- table-state: ... -->`) without cluttering rendered output.
- **Turnkey Host Serialization:** Provides `mdConfig.tableStateCallback` and `ImGui::ApplyTableStateToMarkdown(...)` to keep raw markdown editor buffers and persistence files in lockstep with user interactions.
- **Continuous Vertical Baseline Alignment for Inline Tags:**
- In `<pill>`, `<badge>`, and `<kbd>`, text rendering anchors directly to the line's natural font baseline (`CursorPos.y + CurrLineTextBaseOffset`), and the background capsule expands symmetrically above and below. Bare text, bold text, and capsule text share a single unbroken typographic baseline without sagging or vertical drift.
- **Centering Layout Primitives (`<center>` and `-> text <-`):**
- Effortlessly center briefing headers, warnings, and status lines via HTML-style `<center>text</center>` (or `<centre>`) tags or ASCII block-level markers (`-> Centered Text <-` and `= Centered Text =`).
- Dynamically pre-measures line advance width across headers, bold/italics, pills, and badges to ensure true horizontal optical centering.
- **Dynamic Font Scaling:**
- Uses `ImGui::GetFontScale()` to cleanly honor parent window text zoom and high-DPI display scales.
- **Custom Tag Registration:**
- Supports custom XML/HTML-style tags (e.g., `<btn>Label</btn>`, `<badge>OK</badge>`) that execute arbitrary C++ rendering callbacks inline with text flow.
- **Collision-Free Scope (`ImGui::MD::`):**
- Scoped under `ImGui::MD::` to mirror standard ImGui vocabulary (`Text`, `TextWrapped`) without colliding with core ImGui functions.

Quick Start

1. Include the Header

```cpp
#include "contrib/extensions/Markdown/imgui_markdown_ext.h"
// (Or simply: #include "contrib.h")
```

IMPORTANT

Migrating from Upstream `imgui_markdown`: Remove Old Includes! If your plugin previously included upstream `imgui_markdown.h` directly: 1. **Remove your old `#include "imgui_markdown.h"`** lines and delete any loose copies of `imgui_markdown.h` from your project's header search paths. 2. **Include `#include "contrib.h"` (or `#include "contrib/extensions/Markdown/imgui_markdown_ext.h"`) instead.** `imgui_markdown_ext.h` automatically includes our modernized, patched engine (`contrib/extensions/Markdown/imgui_markdown.h`) under the hood. Keeping an older upstream copy in your project tree risks compiling against the unpatched upstream engine, which lacks our typographic lookahead, mathematical hanging indents, and `ImGui::MD::` widget suite!

2. Configure Default Styles & Fonts (Optional)

You are responsible for loading your own fonts into your `ImgFontAtlas` during plugin startup (e.g., in `InitializeImGui()`). There is no runtime font synthesization in Dear ImGui—if you want Markdown to render distinct headers, bold text, or italics, you must load those fonts into your atlas.

As demonstrated in **sample-code/InitializeImGui.cpp**, each sequential, non-merged font added increments an index in `ImGui::GetIO().Fonts->Fonts[i]`. We recommend defining macro aliases for these slots:

```cpp
// Example font slot definitions (from sample-code/InitializeImGui.cpp):
#define IM_FONT_SLOT(i)        ImGui::GetIO().Fonts->Fonts[i]
#define IM_NORMAL_FONT         IM_FONT_SLOT(0) // Regular base font (14px)
#define IMG_TITLE_FONT         IM_FONT_SLOT(1) // Large header font
#define IM_SMALLER_FONT        IM_FONT_SLOT(2) // Compact UI font
#define IM_BOLD_FONT           IM_FONT_SLOT(3) // Bold font
#define IM_BOLD_LARGER_FONT    IM_FONT_SLOT(4) // Bold title font
#define IM_MONO_NORMAL_FONT    IM_FONT_SLOT(5) // Monospace code font
#define IM_ITALIC_FONT         IM_FONT_SLOT(6) // Italic font
#define IM_MONO_MEDIUM_FONT    IM_FONT_SLOT(7) // Monospace medium font
#define IM_BOLD_ITALIC_FONT    IM_FONT_SLOT(8) // Bold-italic font
```

During plugin startup, configure your Markdown appearance globally using `ImGui::MD::Config`:

```cpp
ImGui::MD::Config mdConfig;

// Configure semantic fonts directly (no cryptic array indices!):
mdConfig.h1Font         = IM_BOLD_LARGER_FONT; // # H1
mdConfig.h2Font         = IMG_TITLE_FONT;      // ## H2
mdConfig.h3Font         = IM_BOLD_FONT;        // ### H3
mdConfig.boldFont       = IM_BOLD_FONT;        // **Bold** / __Bold__
mdConfig.italicFont     = IM_ITALIC_FONT;      // *Italic* / _Italic_
mdConfig.boldItalicFont = IM_BOLD_ITALIC_FONT; // ***Bold-Italic*** / ___Bold-Italic___
mdConfig.monoFont       = IM_MONO_NORMAL_FONT; // `code` / <code> / <mono>

// Configure tight, clean bullet spacing:
mdConfig.bulletSpacing = 4.0f;

// Set globally for the extension:
ImGui::SetDefaultMarkdownConfig(&mdConfig);
```

NOTE

Graceful Font Fallback: If any font pointer is left as `nullptr`, the engine gracefully falls back: * `h1Font`, `h2Font`, `h3Font` fall back to `headingFormats[0..2]` (or the ambient window font). * `boldFont` falls back to `headingFormats[3]` (or the ambient window font). * `italicFont` falls back to `ImGuiCol_TextDisabled` (upstream's dimmed text color). * `monoFont` falls back to the ambient window font.

3. Rendering Markdown in Windows

Simply call `ImGui::Markdown::Text` (or the shorthand `ImGui::MD::Text`):

```cpp
// Standard single-line or multi-line Markdown:
ImGui::Markdown::Text("### System Status\nEngine 1: **Normal** | Fuel: *Sufficient*");

// Paragraph block with automatic word wrapping:
ImGui::Markdown::TextWrapped(
    "Check this option to enable **custom cursors**. "
    "Changes take effect immediately without requiring a simulator restart."
);

// Clickable Markdown text link:
if (ImGui::Markdown::TextLink("Open **Avionics Settings**")) {
    OpenSettings();
}
```

TIP

Forgiving Aliases: Prefer shorter names? `ImGui::MD::Text(...)` is an official shorthand alias for `ImGui::Markdown::Text(...)`. Root shorthands like `ImGui::TextMD(...)` work directly under `ImGui::` as well.

TIP

`#`, `##`, and `###` Are Pure Markdown Headers (Never Swallowed as ImGui IDs!): In standard Dear ImGui widgets (`ImGui::Button`, `ImGui::Checkbox`, `ImGui::CollapsingHeader`, etc.), `"##"` is reserved as an ID separator that hides everything after `"##"`. In `ImGui::MD::Text(...)`, `ImGui::MD::TextWrapped(...)`, and `ImGui::TimedTooltip::TextMD(...)`, text is rendered via `ImGui::TextUnformatted(start, end)`, bypassing ImGui's `"##"` label parser completely. You can freely start strings or lines with `#`, `##`, or `###` for H1/H2/H3 headers without any escaping!

4. Rendering Markdown in Timed Tooltips

When combined with the companion **TimedTooltip** extension (included in either order, or via `#include "contrib/includes.h"`), rich multi-line Markdown tooltips are attached with a single line of code:

```cpp
#include "contrib/includes.h"

ImGui::Button("Accessibility Mode");
ImGui::TimedTooltip::TextMD(
    "### Accessibility Options\n"
    "Enables **larger fonts** and high-contrast styling across all plugin chrome.\n\n"
    "* Fast toggle: `Ctrl+Shift+A`\n"
    "* Requires zero reload."
);
```

Configuration Reference (`ImGui::MD::Config` / `ImGui::MarkdownConfig`)

For clean namespace symmetry with `ImGui::TimedTooltip::Config`, the configuration struct is accessible as both `ImGui::MD::Config` and `ImGui::MarkdownConfig`.

Field	Type	Default	Description
`boldFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for bold emphasis (`**bold**` / `__bold__`). (Falls back to `headingFormats[3].font`.)
`italicFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for true italics (`*italic*` / `_italic_`). (Falls back to `ImGuiCol_TextDisabled` if null.)
`monoFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for monospace code spans (` code ) and **[mono] / [code]**` tags. (Falls back to ambient font if null.)
`h1Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H1 headers (`# Header`). (Falls back to `headingFormats[0].font`.)
`h2Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H2 headers (`## Header`). (Falls back to `headingFormats[1].font`.)
`h3Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H3 headers (`### Header`). (Falls back to `headingFormats[2].font`.)
`bulletSpacing`	`float`	`0.0f` (`4.0f` active)	Horizontal gap in pixels between the bullet circle glyph and the following text.
`codeBlockCallback`	`MarkdownCodeBlockCallback *`	`nullptr`	Optional host callback to customize syntax coloring, lexing, and custom token layout for fenced code blocks.
`codeBlockDescriptionCallback`	`const char* (*)(const char*, size_t, void*)`	`nullptr`	Optional host callback returning tooltip documentation for custom or recognized code block languages.
`setClipboardTextFn`	`void (*)(const char*, void*)`	`nullptr`	Optional host callback to forward code block copies directly to OS-level system clipboards (e.g. macOS NSPasteboard).
`showCodeCopyButton`	`bool`	`true`	When true, renders an interactive one-click `[Copy]` button in the code block header bar.
`tabSize`	`int`	`4`	Column-aligned tabstop expansion width for `\t` characters in code blocks.
`headingFormats`	`HeadingFormat[4]`	`{}`	Upstream array fallback for H1 (`[0]`), H2 (`[1]`), H3 (`[2]`), and Bold (`[3]`).
`linkCallback`	`LinkCallback`	`nullptr`	Upstream hook invoked when the user clicks a markdown `[link](url)`.
`tooltipCallback`	`TooltipCallback`	`nullptr`	Upstream hook invoked when hovering a link with tooltip description.

Field	Type	Default	Description
`imageCallback`	`ImageCallback`	`nullptr`	Upstream hook invoked to render inline markdown `![alt](url)` images.
`formatCallback`	`FormatCallback`	`defaultMarkdownFormatCallback`	Low-level upstream hook invoked when entering/exiting format blocks (headers, lists, bold, italic, code).

> **NOTE**
>
> **Do I Need to Implement the Callbacks?** In 99% of UI tasks, **no**. The callbacks (`linkCallback`, `tooltipCallback`, `imageCallback`, and `formatCallback`) are low-level C-style hooks inherited directly from the upstream `enkisoftware/imgui_markdown` base engine. With the semantic font properties (`boldFont`, `italicFont`, `monoFont`, `h1Font`, etc.) and the high-level tag registration API (`ImGui::MD::RegisterTag`), you do not need to write custom format callbacks.

Emphasis & Monospace Typography: Bold, Italics, and Code

Standard Markdown and CommonMark recognize distinct emphasis and code levels:

- **Level 1 Emphasis (`*italic*` or `_italic_`):** Traditionally rendered as italics (`<em>`).
- **Level 2 Emphasis (`**bold**` or `__bold__`):** Traditionally rendered as bold (`<strong>`).
- **Code Spans (`` code `) / [code] / [mono]`):** Rendered in monospace font ([code]`).

1. Bold Emphasis (`**bold**` / `__bold__`)

Bold emphasis automatically uses `mdConfig.boldFont`. If set (e.g. `IM_BOLD_FONT`), all double-delimiter text renders in crisp bold. If left `nullptr`, it checks `headingFormats[3].font` before falling back to the ambient window font.

2. True Italics (`*italic*` / `_italic_`)

In upstream `enkisoftware/imgui_markdown`, there was no field for italics, so single-delimiter text was given a shortcut: tinted with `ImGuiCol_TextDisabled` (a dimmed, half-opacity gray).

With our extension, simply assign your loaded italic font to `mdConfig.italicFont`:

```cpp
mdConfig.italicFont = IM_ITALIC_FONT;
```

When `italicFont` is assigned, `*hello*` and `_hello_` render in true, crisp italic typography rather than dimmed disabled gray. If left `nullptr`, it preserves upstream's disabled-color fallback for compatibility.

3. Monospace Code Spans (`` code `) and Code Tags ([code] / [mono]`)

For code identifiers, hotkeys, file paths, and terminal syntax, assign your loaded monospace font to `mdConfig.monoFont`:

```cpp
mdConfig.monoFont = IM_MONO_NORMAL_FONT;
```

You have two complementary options for rendering monospace text:

- **Inline Backtick Spans (`` code `):** Standard CommonMark inline backticks switch cleanly to mdConfig.monoFont` while preserving the surrounding text color and sentence flow.
- **Code Tags (`<code>foobar</code>` or `<mono>foobar</mono>`):** Switches to `mdConfig.monoFont` and automatically draws a subtle, translucent rounded pill background (`IM_COL32(128, 128, 128, 60)`) behind the text bounding box, providing a sleek styled code badge (similar to GitHub and documentation sites). Accepts optional

background tinting via parameter (e.g. **<code=#334455>foobar</code>**).

4. Multi-Line Fenced Code Blocks (` `lang / ~~~lang `)

For multi-line snippets, function signatures, configuration files, and terminal logs, use standard CommonMark triple backticks (or tildes):

```
[MARKDOWN]
```cpp
void Initialize() {
    ImGui::Badge("ONLINE", ImGui::ImGuiBadgeVariant_Success);
}
```
```

- **Features:**
- **Automatic Monospace Typography:** Uses **mdConfig.monoFont** (or ambient monospace font) line-by-line without word-wrapping or accidental markup parsing.
- **Responsive Width Clamping & Soft-Wrapping:** Dynamically hugs content lines while clamping maximum container width to available window space (**avail_width**). Wrapped continuation lines flow naturally without horizontal overflow.
- **CommonMark 4.5 Indentation Stripping:** When code blocks are nested inside list items or indented sections, leading indentation matching the opening fence is automatically stripped from code content, preventing code from being pushed awkwardly to the right.
- **Indented Closing Fences:** Supports closing fences indented by ≥ 4 spaces or tabs without swallowing subsequent markdown.
- **Language Badge (Steve TextLink):** Renders the language identifier on the left of the header bar. Formats recognized by built-in highlighters (**lua**, **cpp**, **markdown**, **plaintext**) or host callbacks are color-branded (Lua blue, C++ cobalt, Markdown teal, Plain text slate, Custom purple); unrecognized languages are styled in muted neutral charcoal. Hovering triggers **ImGuiMouseCursor_Hand**; clicking toggles an informative persistent tooltip (e.g. **"C++ format (Syntax highlighted)"**).
- **Animated Raw / Preview Toggle:** For **markdown** or **md** containers, renders an animated sliding toggle switch (**Raw [===O] Preview**) with smooth thumb easing, mouse-wheel scrub support, and live re-entrant formatting (including nested child code blocks). In scrollable viewports, mouse-wheel auto-defeat prevents accidental toggles while navigating long documents.
- **One-Click Vector Clipboard Copy:** Integrated procedural vector copy button on the far right with a transient 1.5s **Copied!** visual feedback state, **ImGuiMouseCursor_Hand** hover cursor, and optional host **setClipboardTextFn** forwarding. Scoped via **ImGuiStorage** to ensure multiple code blocks in the same window maintain their feedback independently without cross-talk.
- **Tabstop Expansion:** Universal 4-character column-aligned tabstop expansion (**tabSize = 4**).
- **CommonMark Compliant:** Supports both backticks (` `) and tildes (~~~`) with optional indentation and variable fence lengths.

5. Thematic Breaks / Horizontal Rules (---, *, ___)**

Standard CommonMark 4.1 thematic breaks create visual section dividers:

```
[MARKDOWN]
Section 1 text.
---
Section 2 text.
```

- Recognizes **---**, *******, and **___** with optional spaces between markers or at the end of the line.
- Renders a clean divider using **ImGui::Separator()**.
- Automatically balances spacing so dividers preceding or following headings do not create excessive empty vertical voids or stacked separators.

6. Interactive CommonMark Tables (Sorting, Resizing & Bi-Directional Persistence)

Standard CommonMark pipe tables render as full-featured, interactive Dear ImGui tables:

```
[MARKDOWN]
| Waypoint | Target Alt | Fuel Remaining | Est. Time |
|:---|:---:|---:|:---|
| **SEA8** | 3,000 ft | 98% | 18:05z |
| **JAWBN** | 8,000 ft | 92% | 18:18z |
| **BTG** | 10,000 ft | 86% | 18:32z |
<!-- table-preset: width=450 sort="Waypoint" asc -->
<!-- table-state: width=329 cols=51,72,94,75 sort="Waypoint" asc -->
```

- **Features:**
- **CommonMark Syntax & Cell Alignment:** Parses headers, separator row, and data rows with cell text alignments (`:---` left, `:---:` center, `---:` right).
- **Rich Cell Content:** Cells support full inline Markdown formatting (bold, italic, code spans, **`<pill>`**, **`<badge>`**, etc.).
- **Tristate Header Sorting:** Click any column header to cycle between **`Ascending`** ↑, **`Descending`** ↓, and **`None`** (unsorted). Renders directional arrows and native hand cursor (**`ImGuiMouseCursor_Hand`**).
- **Internal Column Divider Dragging:** Drag any column boundary separator to adjust relative column widths.
- **Rightmost Width Grip Handle:** An interactive grab-handle on the rightmost edge of the table allows stretching or contracting the overall table width up to 5x available window space.
- **Horizontal Scroll Protection:** If the containing window has **`ImGuiWindowFlags_HorizontalScrollbar`**, tables retain their natural readable widths without being squished when the host window is narrow.
- **Double-Click A/B Toggle Memory:** Double-clicking the right border grip toggles back and forth between the baseline preset dimensions and the user's customized geometry (**`cols=`** and **`width=`**), preserving custom column widths across multi-cycle toggling.

Two-Tier Markdown Directives

Table configuration is persisted directly inside the Markdown text via invisible HTML comment chasers docked immediately beneath the table:

1. **Author Baseline Preset (`<!-- table-preset: ... -->` or `<!-- table: default ... -->`):**

- Placed by document authors to define reference geometry and default sort order:

```
[MARKDOWN]
    <!-- table-preset: width=450 sort="Waypoint" asc -->
```

2. **Active User State (`<!-- table-state: ... -->` or `<!-- table: user ... -->`):**

- Generated automatically when a user modifies column widths, total width, or sort column/direction:

```
[MARKDOWN]
    <!-- table-state: width=329 cols=51,72,94,75 sort="Waypoint" desc -->
```

- When double-clicking to reset to baseline, **`width=`** and **`cols=`** are automatically omitted, retaining only non-default sort overrides (e.g. **`<!-- table-state: sort="Waypoint" desc -->`**).

Bi-Directional Host Synchronization

To synchronize live UI interactions (sorting, resizing) back to your raw Markdown buffer or file storage:

```cpp
ImGui::MD::Config mdConfig;

mdConfig.tableStateCallback = [](const ImGui::MarkdownTableState& state, void* userData) {
    auto* editorBuffer = static_cast<std::string*>(userData);

    // Commit only on user release or definitive clicks:
    if (state.isCommitted) {
        *editorBuffer = ImGui::ApplyTableStateToMarkdown(*editorBuffer, state);
    }
};
mdConfig.userData = &myMarkdownString;
```

7. Built-in Image Services (Tiers 1, 2 & 3)

Images in Markdown (`![alt text](url)`) are resolved automatically via `ImgWindow`'s built-in 3-tier image service, requiring zero boilerplate callbacks:

1. **Tier 1: In-Memory Named Textures (`texture://name` or `name`):**

- Pre-register memory textures (cockpit gauges, avionics icons, branding):

```cpp
ImgWindow::RegisterMarkdownNamedTexture("acme_logo", myTexId, ImVec2(64, 64));
```

- Markdown references like `![Logo](texture://acme_logo)` or `![Logo](acme_logo)` resolve instantly with zero disk I/O.

2. **Tier 2: Local Disk Images:**

- Referenced via relative or absolute paths: `![Airplane](imgui_demo.jpg)` or `![Diagram](./Resources/diagram.png)`.
- Automatically searched relative to the active document directory, the calling plugin's directory, and the X-Plane system root.
- Decoded safely via embedded `stb_image` (forcing 4-channel RGBA8 for Metal/Vulkan hardware safety), uploaded via `ImgWindow::CreateTexture()`, and cached in VRAM with negative caching to prevent disk thrashing.

3. **Tier 3: Asynchronous Remote Web Images (`http://`, `https://`):**

- Referenced directly via web URLs: `![Badge](https://img.shields.io/badge/X--Plane-12.00+-blue.png)`.
- **Zero-Stall Sim Loop:** Network downloads and CPU image decompression run completely off-thread in the background, keeping the simulation loop running smoothly at 60+ FPS.
- **Main-Thread GPU Upload:** Decoded pixel buffers are handed off to the main render thread and uploaded via `ImgWindow::CreateTexture()` without blocking frame rendering.
- **Transient Loading State:** While in flight, the renderer displays a subtle disabled indicator (`( Loading https://... )`) which seamlessly converts to the rendered image once ready.
- **Zero External Dependencies:** Dynamically resolves system `libcurl` on macOS/Linux and native `WinINet` on Windows without requiring any downstream plugins to modify their linker flags or `CMakeLists.txt`.
- **Custom Fetch Overrides:** Host plugins with dedicated HTTP managers can register a custom downloader via `ImgWindow::SetMarkdownRemoteFetchCallback(...)`.

8. Image Sizing, Auto-Constraints & Treatments

Images in Markdown support rich layout modifiers, responsive viewport constraints, and decorative treatments:

1. **Auto-Constraint to Viewport Width (`max-width: 100%; height: auto`):**

- High-resolution graphics or photos that exceed the current window or column width (`ImGui::GetContentRegionAvail().x`) are automatically downscaled proportionally, preventing unwanted horizontal scrollbars or clipped avionics layouts.

2. **Explicit Pandoc / Extended Markdown URL Sizing:**

- **Explicit Dimensions:** `![Diagram](diagram.png =300x200)` renders at 300x200 px.
- **Proportional Width:** `![Logo](logo.png =150x)` renders at 150px wide with height scaled to preserve natural aspect ratio.
- **Proportional Height:** `![Badge](badge.png =x30)` renders at 30px high with proportional width.
- **Viewport Percentage:** `![Banner](banner.png =50%)` dynamically renders at 50% of available window width with proportional height.
- **Angle-Bracket CommonMark URLs:** `![Banner](<https://example.com/banner.png =50%>)` or `![Pic](<https://example.com/pic.png> =200x150)`.

3. **Pandoc Attribute Block Syntax:**

- Append curly attributes after the image link:

```
[MARKDOWN]
    ![Cockpit](panel.png){ width=250px height=180px border=#4a90e2 }
```

4. **HTML `<img ... />` Element Support:**

- Supports standard HTML tags, self-closing tags, and void elements:

```
[HTML]
    <img src="https://example.com/badge.png" width="180" height="40" border="#00ff00" />
    <img src="gauge.png" width="50%" border />
```

5. **Image Treatments (Borders & Frames):**

- Add a border by specifying `border` (subtle default frame), `border=red`, or hex colors `border=#4a90e2` across any syntax variant. Cleanly mapped to `ImGui::Image()`'s native border parameter.

6. **VRAM Texture Cache Deduplication:**

- Sizing and treatment modifiers are parsed and stripped *before* cache lookup and network fetch. Multiple markdown instances referencing the same URL with different sizes or borders reuse the exact same underlying GPU texture in VRAM.

> **IMPORTANT**
>
> **Operating System Clipboard Integration Across Platforms:** * **macOS (`APL=1` / `__APPLE__`):** Upstream Dear ImGui compiles with OS clipboard support disabled unless `#define IMGUI_ENABLE_OSX_DEFAULT_CLIPBOARD_FUNCTIONS` is defined in your `imconfig.h` (and `-framework ApplicationServices` linked). If unconfigured, the `[Copy]` button automatically renders as a disabled button with an on-hover guidance tooltip. * **Windows (`IBM=1` / `_WIN32`):** OS clipboard works out of the box via Dear ImGui's built-in Win32 implementation (`user32.lib`). * **Linux (`LIN=1` / `__linux__`):** Upstream Dear ImGui does not provide an out-of-the-box X11 clipboard implementation. To enable system copying on Linux, assign `MarkdownConfig::setClipboardTextFn` or set `ImGui::GetPlatformIO().Platform_SetClipboardTextFn`.

Per-Call Configuration Overrides

Every rendering function in `ImGui::MD` accepts an optional `const MarkdownConfig*` pointer if a specific widget requires unique fonts, callbacks, or bullet spacing:

```
[CPP]
ImGui::MD::Config customConfig = *ImGui::GetDefaultMarkdownConfig();
customConfig.bulletSpacing = 6.0f;

ImGui::MD::Text(markdownString, &customConfig);
```

Layout Pre-Calculation (`CalcTextSize`)

When building stationary tooltips or sizing dynamic OS windows to fit a block of documentation, you need to know the rendered height *before* drawing:

```cpp
ImGuiID widgetId = ImGui::GetID("doc_section");
float wrapWidth = 400.0f; // Constrain to 400px wide
float fontScale = 0.0f;   // 0.0f = inherit ambient scale

// Pre-calculates exact ImVec2 dimensions without rendering to the screen:
ImVec2 requiredSize = ImGui::MD::CalcTextSize(widgetId, 1, docString, wrapWidth, fontScale);

// Use the size to configure window bounds or child frames:
ImGui::SetNextWindowSize(requiredSize);
```

Markdown Styling Tags (`<tag=param>` / `<tag param>`)

The extension includes a built-in pre-processor that allows you to embed rich, inline styling tags and interactive widgets directly within Markdown text without breaking standard Markdown formatting rules:

```markdown
This option is <color=red>highly experimental</color>!
Please verify <backdrop=yellow>active route</backdrop> before flight.
System status: <badge=danger>FAULT</badge> | <badge=success>ONLINE</badge>
```

1. Flexible Tag Syntax

Tags support both HTML-style attributes and key-value formats:

- **Assignment format:** `<tag=red>text</tag>`, `<backdrop=yellow>text</backdrop>`
- **Space-separated format:** `<tag red>text</tag>`, `<color #00FFCC>text</color>`
- **Multi-parameter format:** `<tag=red,bold>text</tag>`, `<tag red bold>text</tag>`

2. Bold, Italic, and Monospace Styling Inside Tags

You can style tag text in two natural ways:

A. Markdown Syntax Inside the Tag (Recommended)

Place standard Markdown delimiters directly inside the tag content:

```markdown
Check <tag red>**accessibility**</tag> or options.
Please review <backdrop=yellow>**critical checklist**</backdrop> before departure.
Click <btn>*more info*</btn> for details.
<pill orange>` □ `</pill>
```

- The engine detects inner Markdown wrappers (`**bold**`, `__bold__`, `*italic*`, `_italic_`, `` ` `` mono `), strips the wrapper characters so delimiters never render literally, and pushes the active bold, italic, or monospace (`mdConfig.monoFont`) font.
- Intentional leading/trailing padding spaces (both outside and inside the delimiters, as well as UTF-8 non-breaking spaces `U+00A0`) are preserved so you can easily pad single-icon pills (e.g., `"<pill orange mono> " ICON_FA_RETWEET " </pill>"` or `"<pill orange> " ICON_FA_RETWEET " </pill>"`).
- For `<backdrop>` and `<pill>`, the font is pushed *before* measuring text width, ensuring the pill bounding box fits the styled glyphs.

B. Tag Parameter Flags

You can also specify **bold** (or **b**), **italic** (or **i**), and **mono** (or **monospace**, **code**, **m**) directly in the tag attributes (alongside optional **color=** / **col=** prefixes):

```
[MARKDOWN]
Check <tag red bold>accessibility</tag> or options.
Check <tag=red,bold>accessibility</tag> or options.
<pill orange mono>  </pill>
<pill color=0xFFA500, mono>  </pill>
<btn bold>Acknowledge</btn>
```

> **NOTE**
>
> **Why `**<tag>...</tag>**` is not supported:** In Markdown and CommonMark, formatting delimiters wrap text spans rather than embedding interactive widgets across boundary tokens. Nesting formatting inside the tag (**<tag>**text****</tag>**) or on the tag (**<tag bold>text</tag>**) follows standard HTML/XML element structure and guarantees correct word wrapping and draw-list grouping.

3. Inline Spacing & Punctuation Flow

Inline tags integrate cleanly into paragraph and sentence flow:

- **Preserved Spaces:** Both inter-word spaces after a closing tag (such as **<tag red>badge</tag> or options**) and intentional padding spaces inside a tag (such as **<pill orange mono> </pill>**) are preserved.
- **Flush Punctuation:** When followed by punctuation without space, such as **<tag red>badge</tag>, options**, the punctuation docks directly flush against the tag border.
- **Automatic Line-Wrap Protection:** If an inline tag, button, or background pill exceeds the remaining horizontal space on the current line, the renderer automatically wraps down to the next line (**EnsureTagFitsOnLine**) to prevent tags from spilling past window borders.

4. Pre-Supplied Built-in Tags (Ready Out-of-the-Box)

The following tags are pre-registered and active by default:

Tag	Syntax Options	Parameters & Variants	Description
color / **col**	**<color=COLOR>text</color>** **<color COLOR [bold] [italic] [mono]>text</color>**	Named colors (**red**, **green**, **blue**, **yellow**, **orange**, **purple**, **violet**, **cyan**, **magenta**, **white**, **black**, **gray**, **gold**, **dark**, **light**) or hex (**#RRGGBB**, **#RRGGBBAA**, **0xRRGGBB**, **0xRRGGBBAA**). Optional flags: **bold**, **italic**, **mono**.	Renders **text** in the specified color and font style.
backdrop / **bg** / **highlight**	**<backdrop=COLOR>text</backdrop>** **<backdrop COLOR [bold] [italic] [mono]>text</backdrop>**	Accepts any named color or hex code. Optional attribute flags: **bold**, **italic**, **mono**.	Renders a rounded background rectangle ("pill") optically centered behind the text bounding box. Backdrops auto-contrast text color against dark backgrounds and never split awkwardly across lines.
badge / **pill** / **tag**	**<badge=VARIANT>text</badge>** **<pill COLOR [bold] [italic] [mono]>text</pill>** **<pill bg=BG fg=FG>text</pill>**	Semantic variants (**danger**, **warning**, **success**, **info**), named colors, or custom hex color. Supports explicit dual-color attributes (**bg=COLOR fg=COLOR**, **color=COLOR text=COLOR**). Optional flags: **bold**, **italic**, **mono**.	Delegates directly to **ImGui::Badge** / **ImGui::Pill** standalone C++ widgets. Renders sleek rounded badges with auto-contrasting text.

Tag	Syntax Options	Parameters & Variants	Description
`kbd` / `key` / `kdb`	`<kbd>Key</kbd>` `<key>Ctrl+C</key>`	Keyboard key or shortcut label (`kdb` is a built-in typo alias).	Delegates directly to `ImGui::Kbd`. Renders a 3D tactile keyboard keycap with bottom bevel/shadow.
`u` / `ins`	`<u>text</u>` `<ins>text</ins>`	Optional color attribute (`<u #FFCC00>`).	Renders text with a pixel-aligned underline rule along the baseline.

`s` / `del` / `strike`	`<s>text</s>`
	`<del>text</del>`
	`<strike>text</strike>`
		Optional color attribute (`<s red>`).	Renders text with a pixel-aligned strikethrough rule across the vertical midline.

`center` / `centre`	`<center>text</center>` `<centre>text</centre>`	Centered block or inline span.	Centers text or multiline blocks horizontally within the available content area. Pre-measures content width across headers, bold/italics, pills, and badges.
`btn`	`<btn=ID>label</btn>` `<btn [bold] [italic] [mono]>label</btn>`	Unique button ID or style flags (`bold`, `italic`, `mono`).	Renders an inline, clickable `ImGui::SmallButton` directly within text flow.
`mono` / `code`	`<mono>text</mono>` `<code [COLOR]>text</code>`	Accepts optional custom background color or hex parameter (and optional `fg=COLOR`).	Renders `text` in monospace (`mdConfig.monoFont`) inside a subtle, translucent rounded background pill.
`small` / `tiny`	`<small>text</small>` `<tiny>micro text</tiny>`	Optional color attribute (`<small #aaa>`).	Renders text using `ImFontRole::Small` (Slot 2, typically 12px) or `ImFontRole::Tiny` (10px micro font or `Small` fallback).
`big` / `large`	`<big>text</big>` `<large>text</large>`	Optional color attribute (`<big orange>`).	Renders text using `ImFontRole::Title` (Slot 1, typically 16px).
`huge`	`<huge>text</huge>`	Optional color attribute (`<huge #ffcc00>`).	Renders text using `ImFontRole::BoldLarge` (Slot 4, typically 16px+ bold).
`regular` / `prop` / `sans`	`<regular>text</regular>` `<sans>text</sans>`	Optional color attribute.	Resets text font back to `ImFontRole::Regular` (Slot 0 base body font) inside mono or heading spans.

(To disable default tags, define `#define IMGUI_DISABLE_MARKDOWN_DEFAULT_TAGS` before including the header.)

Two-Color Pills & Badges (`bg=` and `fg=`)

By default, `<badge>`, `<pill>`, and `<backdrop>` automatically contrast the text color against the background (e.g., using dark text on yellow, light text on dark blue). When you want explicit artistic control over both colors, specify `bg` (background) and `fg` (text/foreground):

```
[MARKDOWN]
Status: <pill bg=warning fg=black>CAUTION</pill>
Release: <badge bg=#1a3a5a fg=#40c0ff bold>v2.2.1</badge>
Dual-tone: <pill purple white italic>SPECIAL</pill>
```

Inner Markdown vs. Tag Attributes

It's helpful to distinguish how formatting reaches your tags:

1. **Inner Markdown (`<tag>**text**</tag>`):**

Works universally across **all** tags (both built-in tags and your own custom registered tags). If you write standard Markdown delimiters inside a tag, the engine automatically pushes the matching bold/italic font and strips the delimiter characters *before* calling the tag callback.

2. **Tag Attributes (`<tag param>text</tag>`):**

Built-in tags (`badge`, `backdrop`, etc.) inspect the opening tag's attribute string for colors and layout flags. When writing custom tags, the raw parameter string is passed directly into your callback as the second argument (`param`), allowing you to build arbitrary domain-specific markup (e.g., `<item id=42 count=3>Widget</item>`).

---

5. Custom Tag Registration

You can register custom semantic tags using three different approaches depending on how much declarative convenience or low-level programmatic control you need:

5.1 Declarative Style Tags (`ImGui::RegisterMarkdownStyleTag` / `ImGui::MD::RegisterStyleTag`)

Quickly bundle colors, fonts, emphasis, and backgrounds into a reusable semantic tag without writing any callbacks or boilerplate:

```
[CPP]
// Green bold text for camera parameters:
ImGui::RegisterMarkdownStyleTag("cam", "green bold");

// Monospace on a 25% translucent gray backdrop (exact match for subtle pill look):
ImGui::RegisterMarkdownStyleTag("viewmode", "mono bg=#88888840");

// Gold monospace on dark-amber highlight:
ImGui::RegisterMarkdownStyleTag("opt", "gold mono bg=#332a00");
```

In your Markdown text:

```
[MARKDOWN]
Adjust camera <cam>pitch</cam> and <cam>yaw</cam> before entering <viewmode>Circle view</viewmode>.
```

Attributes passed at the call-site merge with the default style:

```
[MARKDOWN]
Alert: <cam red>heading</cam> misalignment!
```

5.2 Declarative Template / Alias Tags (`ImGui::RegisterMarkdownTagAlias` / `ImGui::MD::RegisterTagAlias`)

Expands any arbitrary Markdown or XML template. The `%s` token is replaced with the inner text anywhere in the template (and can appear multiple times). Standard Markdown formatting (`**bold**`, `*italic*`, ` code `) inside the template is parsed and rendered automatically!

```
[CPP]
// Wrap inner text with tags and icons:
ImGui::RegisterMarkdownTagAlias("tip", "<style yellow><b>Tip:</b></style> %s");

// Use standard Markdown syntax directly:
ImGui::RegisterMarkdownTagAlias("hotkey", "**[<style #40c0ff mono>%s</style>]**");

// Expand into a styled status pill:
ImGui::RegisterMarkdownTagAlias("axis", "<pill color=#2a6f97>Axis: %s</pill>");
```

In your Markdown text:

```
[MARKDOWN]
<tip>Press <hotkey>Shift+C</hotkey> to reset <axis>Roll</axis>.</tip>
```

5.3 Programmatic Callback Tags (`ImGui::MD::RegisterTag` / `ImGui::RegisterMarkdownTag`)

When you need complete low-level control to invoke arbitrary ImGui widgets, custom drawlist primitives, or dynamic logic:

```
[CPP]
// 1. Parameterized tag: <status=critical>Engine 1</status> or <status ok>Engine 2</status>
ImGui::MD::RegisterTag("status", [](const std::string& text, const std::string& param) {
    ImVec4 col = (param == "critical") ? ImVec4(1.0f, 0.2f, 0.2f, 1.0f) : ImVec4(0.2f, 0.8f, 0.2f, 1.0f);
    ImGui::PushStyleColor(ImGuiCol_Text, col);
    ImGui::TextUnformatted(text.c_str());
    ImGui::PopStyleColor();
});

// 2. Simple single-argument tag: <notice>Check checklists</notice>
ImGui::MD::RegisterTag("notice", [](const std::string& text) {
    ImGui::BulletText("%s", text.c_str());
});
```

TIP

A Friendly Note on C++ Lambdas: If you are new to modern C++, the `[](const std::string& text, const std::string& param) { ... }` syntax is standard C++11 lambda notation for an inline anonymous function. The lambda receives whatever text was enclosed between `` and `` as `text`, and any parameters or attributes from the opening tag as `param`.

(Note: `ImGui::RegisterMarkdownTag` and `ImGui::RegisterMarkdownWidget` are also preserved as direct backward-compatible aliases.)

API Summary

```
[CPP]
namespace ImGui {

    // Global Configuration & Tag Registry:
    void SetDefaultMarkdownConfig(const MarkdownConfig* config);
    const MarkdownConfig*& GetDefaultMarkdownConfig();
    void RegisterMarkdownTag(const std::string& tag_name, TagCallback callback);
    void RegisterMarkdownWidget(const std::string& tag_name, CustomTagCallback callback);
    void RegisterMarkdownStyleTag(const std::string& tag_name, const std::string& default_style_params);
    void RegisterMarkdownTagAlias(const std::string& tag_name, const std::string& format_template);

    namespace MD {
        // Configuration Alias:
        using Config = MarkdownConfig;

        // Custom Tag Registration (Convenience Aliases):
        void RegisterTag(const std::string& tag_name, std::function<void(const std::string& text, const std::str
        void RegisterTag(const std::string& tag_name, std::function<void(const std::string& text)> callback);
        void RegisterStyleTag(const std::string& tag_name, const std::string& default_style_params);
        void RegisterTagAlias(const std::string& tag_name, const std::string& format_template);
```

```
        // Standard Text Rendering (Mirroring standard ImGui vocabulary):
        // - TextUnformatted / Text: Do not wrap by default (unless wrap_width / wrap_size or PushTextWrapPos is supplied)
        // - TextWrapped: Wraps automatically to GetContentRegionAvail().x (or explicit wrap_width / wrap_size)
        void TextUnformatted(const char* text, const char* text_end = nullptr, const MarkdownConfig* config_override = nullptr);
        void TextUnformatted(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        void Text(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        void Text(const std::string& markdown_text, float wrap_width, const MarkdownConfig* config_override = nullptr);
        void Text(const std::string& markdown_text, const ImVec2& wrap_size, const MarkdownConfig* config_override = nullptr);
        void TextWrapped(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        void TextWrapped(const std::string& markdown_text, float wrap_width, const MarkdownConfig* config_override = nullptr);
        void TextWrapped(const std::string& markdown_text, const ImVec2& wrap_size, const MarkdownConfig* config_override = nullptr);

        // Clickable Markdown Text Links (Hand cursor on hover; toggles TimedTooltip on click if return value is ignored)
        LinkClickResult TextUnformattedLink(const char* text, const char* text_end = nullptr, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextUnformattedLink(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextLink(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextLink(const std::string& markdown_text, float wrap_width, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextLink(const std::string& markdown_text, const ImVec2& wrap_size, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextWrappedLink(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextWrappedLink(const std::string& markdown_text, float wrap_width, const MarkdownConfig* config_override = nullptr);
        LinkClickResult TextWrappedLink(const std::string& markdown_text, const ImVec2& wrap_size, const MarkdownConfig* config_override = nullptr);

        // Backward-Compatibility Aliases:
        void TextMD(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);
        void TextWrappedMD(const std::string& markdown_text, const MarkdownConfig* config_override = nullptr);

        // Layout Pre-Measurement:
        ImVec2 CalcSize(ImGuiID id, int pass, const std::string& text, float wrap_width = 0.0f, float font_scale = 0.0f, const
        ImVec2 CalcTextSize(ImGuiID id, int pass, const std::string& text, float wrap_width = 0.0f, float font_scale = 0.0f, b
        ImVec2 CalcMarkdownSize(ImGuiID id, int pass, const std::string& text, float wrap_width = 0.0f, float font_scale = 0.0

        // Canonical Markdown Button Widget (1:1 Drop-in Mirror of ImGui::Button):
        bool Button(const char* label, const ImVec2& size = ImVec2(0, 0));
        bool Button(const std::string& label, const ImVec2& size = ImVec2(0, 0));
        bool Button(const char* label, const char* tooltip_md, float tooltip_wrap_width = 400.0f, bool tooltip_allowed = true
        bool Button(const std::string& label, const std::string& tooltip_md, float tooltip_wrap_width = 400.0f, bool tooltip_

        // Backward-Compatibility & Delayed Tooltip Aliases:
        bool ButtonMD(const char* label, const ImVec2& size = ImVec2(0, 0));
        bool ButtonMD(const std::string& label, const ImVec2& size = ImVec2(0, 0));

        bool ButtonMD(const char* label, const char* tooltip_md, float tooltip_wrap_width = 400.0f, bool tooltip_allowed = tr
        bool ButtonMD(const std::string& label, const std::string& tooltip_md, float tooltip_wrap_width = 400.0f, bool toolti
        bool DelayedTooltipMD(const char* markdown_text, float wrap_width = 400.0f, int delay_frames = 30, bool tooltip_allo
        bool DelayedTooltipMD(const std::string& markdown_text, float wrap_width = 400.0f, int delay_frames = 30, bool is_all
    }
}

// Top-Level Convenience Aliases (root ImGui:: namespace):
bool ImGui::ButtonMD(const char* label, const ImVec2& size = ImVec2(0, 0));
bool ImGui::ButtonMD(const std::string& label, const ImVec2& size = ImVec2(0, 0));
bool ImGui::ButtonMD(const char* label, const char* tooltip_md, float tooltip_wrap_width = 400.0f, bool tooltip_allowed = true
bool ImGui::ButtonMD(const std::string& label, const std::string& tooltip_md, float tooltip_wrap_width = 400.0f, bool toolti
bool ImGui::DelayedTooltipMD(const char* markdown_text, float wrap_width = 400.0f, int delay_frames = 30, bool tooltip_allo
bool ImGui::DelayedTooltipMD(const std::string& markdown_text, float wrap_width = 400.0f, int delay_frames = 30, bool is_all
```

Interactive Buttons (`ImGui::Markdown::Button` / `ImGui::ButtonMD`)

`ImGui::Markdown::Button` (aliased as `ImGui::MD::Button` and `ImGui::ButtonMD`) is the **1:1 drop-in Markdown equivalent of Dear ImGui's native `ImGui::Button()`**. It accepts the exact same `(label, size)` signature in both `const char*` and `std::string`, but parses and renders the face using the Markdown engine.

- **Standalone Pill & Badge Buttons:**

When passed a `<pill>` or `<badge>` tag (e.g. `"<pill orange> " ICON_FA_RETWEET " </pill>##test"`), it renders the styled rounded pill or badge directly and overlays an `InvisibleButton` with hand cursor. **Zero gray button box drawn behind your pill.**

- **Rich Markdown Text on Button Faces:**

Labels with **`**bold**`**, **`_italic_`**, **`<col red>Warning</col>`**, or **`<code>mono</code>`** render the native ImGui button frame with the multi-formatted Markdown text centered inside.

- **Vector Icon Button Faces:**

If the button label includes an icon tag (**`<icon caret-up />`**), an emoji shortcode (**`:undo: Reset`**), raw Unicode (**`▲##up`**), or Steven's raw-fallback link macro (**`#[^](ICON_CARET_UP)`**), it renders the procedural vector icon directly on the button face.

- **Pure icon buttons** (e.g. **`Button("<icon caret-up />##up")`**) default to square geometry matching **`ImGui::GetFrameHeight()`**.
- **Icon + Text buttons** (e.g. **`Button(":gear: Settings")`** or **`Button("<icon undo color=#ff5555 /> Revert")`**) auto-size to fit both the icon and text with standard inner item spacing.
- **No font dependencies:** Never displays missing-glyph diamond boxes (**`?`**).
- **Integrated Timed Tooltip Engine:**

The **`tooltip_md`** overloads in **`Button(...)`** / **`ButtonMD(...)`** and the companion **`DelayedTooltipMD(...)`** function automatically route through the stationary **`TimedTooltip`** engine (when available), rendering rich markdown sticky notes with dwell delays, anti-jitter positioning, and screen boundary clamping rather than popping unformatted legacy tooltips.

- **Hand Cursor:**

Automatically activates **`ImGuiMouseCursor_Hand`** when hovered.

```cpp
// 1. Standalone Pill Button (no gray background, clean oval clickable pill):
if (ImGui::Markdown::Button("<pill orange> " ICON_FA_RETWEET " </pill>##test_btn", ImVec2(32, 20))) {
    ToggleWidgetTest();
}

// 2. Pure icon button (square, auto-sized to GetFrameHeight()):
if (ImGui::ButtonMD("<icon caret-up />##scroll_top", "Scroll to top of document")) {
    ScrollToTop();
}

// 3. Rich Markdown Button (bold + regular text):
if (ImGui::ButtonMD("**Save** Changes", ImVec2(120, 24))) {
    SaveChanges();
}

// 4. Raw Unicode rescue:
if (ImGui::ButtonMD("▲##zoom_in", "Increase preview zoom")) {
    ZoomIn();
}

// 5. Icon + Text with custom icon color and Markdown tooltip:
if (ImGui::ButtonMD("<icon undo color=#ff5555 /> Reset All##reset", "**Warning:** Reverts *all* preferences to f
    ResetDefaults();
}
```

Appendix: Real-World Example & Comparison

Below is a complete real-world example illustrating how the **Markdown** and **TimedTooltip** extensions combine to create a rich, responsive settings and reference panel with custom tags, nested list items, and stationary tooltips.

Side-by-Side: Why Use the Extensions?

To appreciate what the extensions handle behind the scenes, consider what is required to build a mixed-format settings card with highlight backdrops, colored status pills, bullet lists with hanging indents, and a stationary clamped tooltip:

The Hard Way: Vanilla Dear ImGui (Without Extensions)

[CPP]

```cpp
// ---------------------------------------------------------------------------
// VANILLA IMGUI: ~70+ lines of low-level boilerplate, manual geometry math,
// and state storage just for a single header, backdrop pill, and timed tooltip!
// ---------------------------------------------------------------------------
void RenderSettingsPanelVanilla()
{
    ImGui::Begin("Plugin Settings & Reference");

    // 1. Header: Manual font push, separator, and manual pop
    ImGui::PushFont(IMG_TITLE_FONT);
    ImGui::TextUnformatted("Welcome to Avionics Manager");
    ImGui::PopFont();
    ImGui::Separator();

    // 2. Mixed Bold Text + Background Highlight Pill:
    // In vanilla ImGui, you cannot wrap text across lines if you embed custom drawn rectangles.
    // You have to manually calculate draw list coordinates, text sizes, and padding:
    ImGui::TextUnformatted("Configure your autopilot, navigation modes, and ");
    ImGui::SameLine(0.0f, 0.0f);

    ImVec2 p0 = ImGui::GetCursorScreenPos();
    ImVec2 text_sz = IM_BOLD_FONT->CalcTextSizeA(IM_BOLD_FONT->FontSize, FLT_MAX, 0.0f, "expert features");
    ImVec2 p1 = ImVec2(p0.x + text_sz.x + 8.0f, p0.y + text_sz.y + 2.0f);

    // Draw background pill manually:
    ImGui::GetWindowDrawList()->AddRectFilled(p0, p1, IM_COL32(255, 230, 60, 255), 4.0f);

    // Position text inside pill and draw in dark contrast color:
    ImGui::SetCursorScreenPos(ImVec2(p0.x + 4.0f, p0.y + 1.0f));
    ImGui::PushFont(IM_BOLD_FONT);
    ImGui::PushStyleColor(ImGuiCol_Text, IM_COL32(20, 20, 20, 255));
    ImGui::TextUnformatted("expert features");
    ImGui::PopStyleColor();
    ImGui::PopFont();

    // Resume following text:
    ImGui::SetCursorScreenPos(ImVec2(p1.x, p0.y));
    ImGui::TextUnformatted(" below.");

    ImGui::Spacing();

    // 3. Bullet List:
    // Vanilla ImGui::BulletText does NOT support hanging indents when text wraps.
    // Continuation lines wrap directly beneath the bullet circle instead of indenting cleanly.
    // Fixing this requires manual ImGui::Indent() / ImGui::Unindent() bookkeeping.
    ImGui::Bullet();
    ImGui::SameLine(0.0f, 4.0f);
    ImGui::TextWrapped("autopilot: Controls automated flight guidance (LNAV, VNAV, FLCH).");

    ImGui::Spacing();

    // 4. Reset Button with Stationary Clamped Tooltip:
    // Vanilla ImGui::SetTooltip moves with the mouse, has no hover delay, and lacks screen clamping.
    // Building a stationary tooltip with a 1-second delay and viewport boundary clamping requires
    // querying frame counters, ImGuiStorage, and calculating display boundary math:
    ImGui::Button("Reset to Defaults");

    static int s_HoverFrames = 0;
    if (ImGui::IsItemHovered()) {
        s_HoverFrames++;
```

```
        if (s_HoverFrames > 60) { // ~1.0 second delay
            ImVec2 mousePos = ImGui::GetIO().MousePos;
            ImVec2 dispSize = ImGui::GetIO().DisplaySize;
            ImVec2 tipSize(260.0f, 80.0f); // Guessed height!

            // Manual screen boundary clamping:
            ImVec2 tipPos = ImVec2(mousePos.x, mousePos.y + 20.0f);
            if (tipPos.y + tipSize.y > dispSize.y) tipPos.y = mousePos.y - tipSize.y - 10.0f;
            if (tipPos.x + tipSize.x > dispSize.x) tipPos.x = dispSize.x - tipSize.x - 10.0f;

            ImGui::SetNextWindowPos(tipPos);
            ImGui::PushStyleColor(ImGuiCol_PopupBg, IM_COL32(255, 242, 153, 255));
            ImGui::PushStyleColor(ImGuiCol_Text, IM_COL32(0, 0, 0, 255));
            if (ImGui::BeginTooltip()) {
                ImGui::TextUnformatted("Restores all settings to factory defaults.");
                ImGui::EndTooltip();
            }
            ImGui::PopStyleColor(2);
        }
    } else {

        s_HoverFrames = 0;
    }

    ImGui::End();
}
```

The Clean Way: With ImgWindow Extensions (`MD` + `TimedTooltip`)

[CPP]

```
// ---------------------------------------------------------------------------
// WITH EXTENSIONS: Clean, declarative, expressive code.
// The engine automatically handles dynamic font scaling, lookahead word wrapping,
// hanging bullet indents, draw-list pills, delay timers, and screen docking!
// ---------------------------------------------------------------------------
#include "contrib/extensions/Markdown/imgui_markdown_ext.h"
#include "contrib/extensions/TimedTooltip/imgui_tooltip_ext.h"

void RenderSettingsPanelExtensions()
{
    ImGui::Begin("Plugin Settings & Reference");

    // 1. Rich Header and Overview with Custom Backdrop Pill and Tags:
    ImGui::MD::TextWrapped(
        "### Welcome to Avionics Manager\n"
        "Configure your autopilot, navigation modes, and <backdrop=yellow>**expert features**</backdrop> below.
        "Hover over any setting or <tag red>tag</tag> for operational manuals."
    );

    ImGui::Separator();

    // 2. Multi-Line List with Nested Bullets, Badges, and Automatic Hanging Indents:
    ImGui::MD::TextWrapped(
        "* <tag red>modes</tag>: _Click_ on active tags above to filter the feature catalog.\n"
        "  * _Important_: Filter queries use exact \"<tag yellow>substring</tag>\" matching.\n"
        "  * Toggle back to preset: `__#1__` / \"**Default Nav**\" <pill green>Active</pill>.\n"
        "* <tag blue>autopilot</tag>: Controls automated lateral and vertical flight guidance.\n"
        "  * Supported modes: **LNAV**, **VNAV**, and **FLCH**.\n"
        "  * Disconnect shortcut: `Ctrl+Shift+D`."
    );

    ImGui::Spacing();

    // 3. Interactive Widgets with Stationary Clamped Markdown Timed Tooltip:
    if (ImGui::Button("Reset to Defaults")) {
        // Reset logic...
    }
    // Automatically binds to the preceding button, manages hover cycle delays,
    // isolates styles, clamps to screen borders, and renders rich Markdown:
    ImGui::TimedTooltip::TextMD(
```

```
        "### Reset Configuration\n"
        "Restores all plugin settings back to factory defaults.\n\n"
        "* **Reversible:** A backup of your current setup is saved automatically.\n"
        "* Active theme: #3/\"**Black & White**\" <pill green>PG</pill>.\n"
        "* *Click to confirm.*"
    );

    ImGui::End();
}
```

ImGee Typography & StyledText Extension

The **Typography** extension provides semantic font hierarchy management and a lightweight, inline rich text engine for Dear ImGui within the **ImGee** suite for **ImgWindow**.

Unlike full Markdown documents (which enforce paragraph margins, headers, block indentation, and thematic breaks), the **Inline StyledText Engine** is designed for tight cockpit UI elements: data readouts, status pills, tables, buttons, and mixed-style sentences where continuous baseline alignment is paramount.

■ Key Features

1. **Semantic Font Hierarchy (`ImFontRole`)**:

Eliminates hardcoded font pointers (`m_pHeaderFont->...`). Plugins register fonts by semantic role (`Title`, `Header`, `Body`, `Small`, `Mono`), and widgets request fonts by role.

2. **Commutative Scoped Font Guard (`ScopedFontRole`)**:

Zero-leak RAII guard that safely pushes a semantic font role and pops it upon exiting the scope.

3. **Inline StyledText Engine (`ImGui::StyledText`)**:

Renders mixed bold, italic, monospace, colored text, badges, pills, keyboard keycaps, and images within a single immediate-mode call.

4. **Continuous Vertical Baseline Alignment**:

Inline badges, pills, `<kbd>` keycaps, and icons automatically compute font metrics (`Font->Ascent`, `Font->Descent`) to align their visual center with the surrounding text baseline, completely eliminating vertical text jitter.

5. **Universal Procedural Vector Icons**:

Full integration with `imgui_icons.h`—supports `<icon caret-up />`, emoji shortcodes (`:gear:`), raw Unicode glyphs (▲, ↻), and named memory textures.

6. **Layout Pre-Measurement (`ImGui::CalcTextStyledSize`)**:

Calculates exact rendered dimensions in pixels without drawing to the screen.

---

1. Semantic Font Roles (`imgui_font_roles.h`)

The Problem

Traditional Dear ImGui plugins pass raw `ImFont*` pointers across classes or query global singletons (`m_Fonts[2]`). If a specific font size or style isn't loaded, code either asserts or crashes.

The Solution: Semantic Roles & 4-Tier Fallback

Font roles decouple your layout code from font assets:

```cpp
#include "contrib/extensions/Typography/imgui_font_roles.h"

// Available Roles:
enum class ImFontRole {
    Default,
    Body,
    Title,
    Header,
    Heading,
    Subheading,
    Caption,
    Small,
    Mono,
    Code,
    Large,
    Bold,
    Italic
};
```

Registering Roles in `ImgFontAtlas`

During plugin startup, bind loaded fonts to semantic roles:

```cpp
void InitializeFonts() {
    auto atlas = ImgWindow::sFontAtlas;
    ImFont* fontRegular = atlas->AddFontFromFileTTF("fonts/Roboto-Regular.ttf", 15.0f);
    ImFont* fontBold    = atlas->AddFontFromFileTTF("fonts/Roboto-Bold.ttf", 15.0f);
    ImFont* fontHeader  = atlas->AddFontFromFileTTF("fonts/Roboto-Bold.ttf", 20.0f);
    ImFont* fontMono    = atlas->AddFontFromFileTTF("fonts/SourceCodePro-Regular.ttf", 14.0f);

    atlas->SetFontRole(ImFontRole::Body,   fontRegular);
    atlas->SetFontRole(ImFontRole::Bold,   fontBold);
    atlas->SetFontRole(ImFontRole::Header, fontHeader);
    atlas->SetFontRole(ImFontRole::Mono,   fontMono);
}
```

Scoped Font Guards (`ScopedFontRole`)

Use RAII guards to push and pop font roles safely:

```cpp
{
    ScopedFontRole headerGuard(ImFontRole::Header);
    ImGui::Text("Avionics Status & Navigation");
} // Restores previous font automatically

{
    ScopedFontRole monoGuard(ImFontRole::Mono);
    ImGui::Text("RAW NMEA: $GPGGA,123519,4807.038,N,01131.000,E*47");
}
```

2. Inline StyledText Engine (`imgui_styled_text.h`)

`ImGui::StyledText` parses lightweight XML-style tags and renders continuous text flows across lines.

Function Signatures

Like standard `ImGui::Text()`, `StyledText` uses `printf`-style variadic format strings:

```cpp
void ImGui::StyledText(const char* fmt, ...);
void ImGui::StyledTextWrapped(const char* fmt, ...);
void ImGui::StyledTextUnformatted(const char* text, const char* text_end = nullptr);
```

Basic Usage

```cpp
#include "contrib/extensions/Typography/imgui_styled_text.h"

// Inline bold, italic, and color:
ImGui::StyledText("Transponder is <color=green>ACTIVE</color> on <b>XPDR1</b>.");

// Variadic formatted output:
ImGui::StyledText("Altitude: <huge>%.0f</huge> <small>FT</small> | Status: <color=green>%s</color>", alt, status

// Wrapped text:
ImGui::StyledTextWrapped("Notice: Route clearance is <i>pending</i> from <color=#3498db>%s</color>.", facility);
```

Supported Tag Catalog

Tag	Example Syntax	Description
Small	`<small>reduced text</small>`	Applies `ImFontRole::Small` (Slot 2, typically 12px / -2px)
Tiny	`<tiny>micro text</tiny>`	Applies `ImFontRole::Tiny` (10px micro font, or fallback to `Small`)
Big / Large	`<big>larger text</big>` or `<large>...</large>`	Applies `ImFontRole::Title` (Slot 1, typically 16px / +2px)
Huge	`<huge>hero text</huge>`	Applies `ImFontRole::BoldLarge` (Slot 4, 16px+ bold)
Regular	`<regular>normal</regular>` or `<sans>...</sans>`	Applies `ImFontRole::Regular` (Slot 0 default body font)
Bold	`<b>bold text</b>` or `<strong>...</strong>`	Applies `ImFontRole::Bold` (Slot 3)
Italic	`<i>italic text</i>` or `<em>...</em>`	Applies `ImFontRole::Italic` (Slot 6)
Monospace	`<mono>code</mono>` or `<code>code</code>`	Applies `ImFontRole::Monospace` (Slot 5)
Color	`<color=red>warning</color>` or `<color=#ffaa00>caution</color>`	Applies foreground text color (named or hex)
Headings	`<h1>Banner</h1>`, `<h2>Section</h2>`, `<h3>Subsection</h3>`	Applies Heading1..3 / Title / Bold font roles
Badges	`<badge=primary>v1.14</badge>`	Inline rounded badge pill
Pills	`<pill color=#2ecc71>ARMED</pill>`	Interactive status pill with background color
Keys	`<kbd>Ctrl+C</kbd>`	3D-styled keyboard keycap
Textures	`<img name="my_logo" width=24 height=24 />`	In-memory texture registered via `ImgWindow::RegisterMemoryImage`
Vector Icons	`<icon caret-up />`, `:gear:`, ▲	Procedural vector icon from `imgui_icons.h`

3. Direct Semantic Text Helpers (`imgui_font_roles.h`)

For standard Dear ImGui UI layouts where XML tags are not needed, direct wrapper functions render text immediately with semantic font roles:

```cpp
#include "contrib/extensions/Typography/imgui_font_roles.h"

// Formatted and unformatted small text (Slot 2, 12px):
ImGui::SmallText("Altitude: %.0f ft", altitude);
ImGui::SmallTextUnformatted("Captions and secondary subtext");

// Micro tiny text (Slot 2 fallback or dedicated 10px):
ImGui::TinyText("v1.0.2 [Beta]");
ImGui::TinyTextUnformatted("FAA Advisory Circular AC 90-66B");

// Title / header text (Slot 1, 16px):
ImGui::TitleText("Flight Plan: %s -> %s", dep, dest);

// Monospace text (Slot 5):
ImGui::MonoText("ICAO: %s | SQUAWK: %04d", callsign, squawk);
```

3. In-Memory Image & Icon Integration

The StyledText engine directly integrates with **`ImgWindow`**'s in-memory texture service and procedural vector icons.

In-Memory Textures (`<img ... />`)

```cpp
// Static embedded PNG array:
static const unsigned char g_BeaconPng[] = { 0x89, 0x50, 0x4e, 0x47, ... };

// 1. Register texture:
int w = 0, h = 0;
ImgWindow::RegisterMemoryImage("avionics_beacon", g_BeaconPng, sizeof(g_BeaconPng), &w, &h);

// 2. Render inline with text:
ImGui::StyledText("Beacon: <img name='avionics_beacon' width=20 height=20 /> ONLINE");

// 3. Embedded inside an icon pill:
ImGui::StyledText("Transponder: <pill icon=avionics_beacon color=#2ecc71>ACTIVE</pill>");

// 4. Standalone unary annunciator:
ImGui::StyledText("Master Warning: <pill icon=avionics_beacon color=#e74c3c />");
```

Procedural Vector Icons (`<icon ... />`)

Zero-font, zero-texture vector icons automatically align with the text baseline:

```cpp
// Explicit icon tag:
ImGui::StyledText("Navigation: <icon caret-up /> Climb to FL350");

// Emoji shortcodes:
ImGui::StyledText("Avionics: :gear: System nominal :check: Verified");

// Raw Unicode rescue:
ImGui::StyledText("Heading: ▲ 180° | Pitch: ▼ -2.5° | Revert: □");
```

4. Pre-Measurement (`CalcTextStyledSize`)

When calculating window bounds, stationary tooltip geometry, or layout columns, compute the exact size without rendering:

```cpp
ImVec2 size = ImGui::CalcTextStyledSize("Transponder: <pill color=green>ACTIVE</pill>", 300.0f);
// size.x = total rendered width in pixels
// size.y = total rendered height in pixels
```

5. Opt-Out Configuration

If your project defines conflicting identifiers, define the following before including **`contrib.h`** or **`extensions.h`**:

```cpp
#define CONTRIB_DISABLE_TYPOGRAPHY    // Disables Typography and StyledText extension
```

ImGee Flight-Deck & Avionics Widgets

This directory contains lightweight, reusable, immediate-mode **Widgets** extending Dear ImGui within the **ImGee** suite for the **ImgWindow** framework.

Unlike **Extensions** (which manage their own complex background subsystems, registries, and configuration structures), **Widgets** are self-contained immediate-mode flight-deck controls. They hook directly into Dear ImGui's ambient context (`GImGui`, ID stack, `ImGuiStorage`, and window draw lists) and live directly in the root `ImGui::` namespace to feel completely native.

Architecture & Inclusion Strategy

1. Root `ImGui::` Namespacing

All widgets in this directory are injected directly into `namespace ImGui { ... }`. You call `ImGui::Knob(...)`, `ImGui::ToggleButton(...)`, `ImGui::SliderWithReset(...)`, `ImGui::Badge(...)`, `ImGui::Icon(...)`, and `ImGui::SelectPulldown(...)` just like Dear ImGui's built-in `ImGui::Checkbox` or `ImGui::SliderFloat`.

2. Inclusion Models

Pattern	Usage	Benefit
Master Contrib Concentrator	`#include "contrib.h"`	Includes the master switchboard which cascades directly into ImGee.
Master ImGee Suite	`#include "contrib/ImGee/imgee.h"`	Includes **all** ImGee extensions (`TimedTooltip`, `Markdown`, `Typography`) and widgets in guaranteed dependency order.
ImGee Widgets Roll-Up	`#include "contrib/ImGee/widgets/includes.h"` *(or `"contrib/widgets.h"`)*	Instant access to the entire widget collection in one line.
A-la-Carte Widget	`#include "contrib/ImGee/widgets/imgui_knobs.h"`	Include only the specific control family you need.

Opt-Out Configuration Flags

If your project defines a symbol with the same name, define any of the following canonical `IMGEE_*` macros *before* including `imgee.h` (legacy `CONTRIB_DISABLE_*` and `IMGUI_DISABLE_*` macro names are also honored automatically):

```[CPP]
#define IMGEE_DISABLE_WIDGETS                // Disables all ImGee widgets
#define IMGEE_DISABLE_KNOBS                  // Disables avionics rotary knobs, dimmer dials & detents
#define IMGEE_DISABLE_MOUSE_WHEEL_CONTROLS   // Disables all wheel sliders, drags, cyclers & combos
#define IMGEE_DISABLE_TOGGLE_BUTTONS         // Disables ToggleButton & ToggleButtons
#define IMGEE_DISABLE_TOGGLE_MOUSE_WHEEL     // Disables mouse-wheel support on ToggleButton
#define IMGEE_DISABLE_BADGES                 // Disables Badge, Pill, AnnunciatorPill & Kbd widgets
#define IMGEE_DISABLE_POPUP_MENUS            // Disables SelectPulldown, OpenPopupAtItem & BeginAnchoredPopup
#define IMGEE_DISABLE_ICONS                  // Disables procedural vector icon engine & ImGui::Icon
#define IMGEE_DISABLE_ICON_BUTTON            // Disables IconButton & ButtonIcon push buttons
#define IMGEE_DISABLE_CLICKABLE_LINKS        // Disables ButtonLink, TextLink & AttachItemCursor
#define IMGEE_DISABLE_SPINNERS               // Disables Spinner, RadialTimer & CountdownTimer
#define IMGEE_DISABLE_COPY_BUTTON            // Disables CopyButton & CopyLastItemButton
#define IMGEE_DISABLE_GET_MOUSE_WHEEL        // Disables GetMouseWheel* primitives
#define IMGEE_DISABLE_ADJUST_MOUSE_WHEEL     // Disables AdjustItemOnMouseWheel()
#define IMGEE_DISABLE_CYCLE_MOUSE_WHEEL      // Disables CycleItemOnMouseWheel()
#define IMGEE_DISABLE_PERCENT_SLIDER         // Disables PercentSlider() & SliderPercent()
#define IMGEE_DISABLE_MOUSE_WHEEL_COMBO      // Disables MouseWheelCombo & ComboWithReset
#define IMGEE_DISABLE_RESET_UNDO             // Disables 1-click Reset-with-Undo (restore previous value)

// Optional compile-time icon & chevron style customization:
```

```
#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL          ICON_FA_UNDO     // Custom primary reset icon (default: vecto
#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT      ICON_FA_REDO     // Custom restore/redo icon (default: vector
#define IMGUI_CONTRIB_DEFAULT_COMBO_ARROW_STYLE     ImGui::ComboArrowStyle_RightWhenClosed // Combo chevron pola
#define IMGUI_CONTRIB_DEFAULT_PULLDOWN_CARET_DOWN  ICON_FA_CHEVRON_DOWN // Custom pulldown down-caret (default:
#define IMGUI_CONTRIB_DEFAULT_PULLDOWN_CARET_UP    ICON_FA_CHEVRON_UP   // Custom pulldown up-caret (default: pr
```

Widget Catalog

1. Toggle Controls (`imgui_toggle_buttons.h`)

A. Animated Toggle Switch (`ImGui::ToggleButton`)

A modern, iOS / Material-design rounded pill switch with a smoothly sliding circular thumb. Accepts both `bool*` and `int*` state variables.

[CPP]

```cpp
#include "contrib/widgets/imgui_toggle_buttons.h"

static bool s_ExpertMode = false;
static int  s_SmoothTransitions = 1;

// 1. Standard switch with clickable label on the right:
ImGui::ToggleButton("Expert Mode", &s_ExpertMode);

// 2. Direct reference syntax (no & required):
ImGui::ToggleButton("Expert Mode", s_ExpertMode);

// 3. Compact switch without visible label (uses ID only):
ImGui::ToggleButton("##hud_switch", &s_HudActive);

// 4. Direct dual-string per-state tooltips (Off tooltip, On tooltip):
ImGui::ToggleButton("HUD Overlay", &s_HudActive,
                    "HUD is currently **OFF**.\n- Click to enable overlay.",
                    "HUD is currently **ON**.\n- Click to disable overlay.");

// 5. Array-based per-state tooltips ({ Off tooltip, On tooltip }):

static const char* const kTransitionTooltips[] = {
    "Smooth transitions are **OFF** (instant snap).\n- Click or scroll **up** on the switch to enable.",
    "Smooth transitions are **ON** (animated).\n- Click or scroll **down** on the switch to disable."
};
ImGui::ToggleButton("Smooth Transitions", &s_SmoothTransitions, ImVec2(0, 0), kTransitionTooltips);

// Optional: Customize switch active track & thumb colors globally:
ImGui::SetDefaultToggleButtonColors(ImVec4(0.26f, 0.59f, 0.98f, 1.0f)); // Active track color
// Or via ImU32:
ImGui::SetDefaultToggleButtonColors(ABCIM_COL32(HILITE_COLOR));
```

- **Features:**
- Frame-rate independent sliding animation powered by `io.DeltaTime` and stored per-widget in `ImGuiStorage`.
- **Flexible Types & Syntax:** Accepts `bool*`, `int*`, `bool&`, or `int&` (`0` = Off, non-zero = On).
- **Global Theme Integration (`SetDefaultToggleButtonColors`):** Easily set the switch track/thumb colors globally via `ImVec4` or `ImU32`. Defaults to your theme's active highlight color (`ImGuiCol_ButtonActive`).
- **Built-In Clamped Mouse-Wheel Support (`allow_mouse_wheel = true`):** Spinning the scroll wheel up/right over the pill switch turns it **On** and clamps there; spinning down/left turns it **Off** and clamps there (never wraps). Hovering over the text label does *not* capture the mouse wheel, preventing accidental toggles when scrolling windows.
- **Per-State Tooltips:** Supports both direct dual-string parameters (`off_tooltip`, `on_tooltip`) and 2-element string arrays (`{ off_tooltip, on_tooltip }`) automatically rendered via `TimedTooltip::TextMDProxy` (or standard `SetTooltip` fallback).
- **Responsive Auto-Scaling:** Switch dimensions scale naturally with the ambient font size (0.78x ambient frame height for standard switches, 0.60x for compact switches), looking crisp on standard, Retina, and 4K displays, with support for

explicit `ImVec2 size` overrides.

- Automatic hand pointer cursor (`ImGuiMouseCursor_Hand` by default, customizable via `hover_cursor` or `IMGUI_EXTRA_TOGGLE_DEFAULT_CURSOR`).

B. Segmented Dual-Button Pair (`ImGui::ToggleButtons`)

Joined side-by-side buttons for explicit binary selections (`Off` / `On`, `Solid Opaque` / `Glass Translucent`). Accepts both `int*` and `bool*`.

```cpp
static int s_WindowStyle = 0; // 0 = Solid Opaque, 1 = Glass Translucent

// 1. Compact pair without a right-side label (starts with "##"):
if (ImGui::ToggleButtons("##style_toggle", &s_WindowStyle, "Solid Opaque", "Glass Translucent")) {
    ApplyStyleChange();
}

// 2. With clickable right-side title ("Surface Style") and wheel-latched hover index:
int hovered_idx = -1;
ImGui::ToggleButtons("Surface Style##style_pair", &s_WindowStyle, "Solid Opaque", "Glass Translucent",
                     ImVec2(130.0f, 0.0f), nullptr /* item_tooltips */, &hovered_idx);
```

- **Features:**
- **Optional Right-Side Clickable Title:** Following standard Dear ImGui `"Title##id"` semantics, if `label` contains text before `"##"`, it is drawn to the right of the button pair, shows a Hand cursor on hover, toggles the state on click, and ignores the mouse wheel.
- **Clamped Mouse-Wheel Toggling:** Scrolling up/right over either button selects `1` (`label_on`); scrolling down/left selects `0` (`label_off`).
- **Per-Button Tooltips (`item_tooltips`) & Wheel-Selection Tooltip Latch (`out_hovered_item`):** When the user spins the mouse wheel over either button to change the state, tooltip targeting (and `*out_hovered_item`) automatically latches onto the newly selected button (`0` or `1`)—even if the stationary mouse cursor is still parked over the unselected button—until the mouse moves (`> 1.5px`).

---

2. Mouse-Wheel Sliders, Drag Boxes, Cyclers & Combos (`imgui_mouse_wheel_controls.h`)

A. Core Mouse-Wheel Primitives (`ImGui::GetMouseWheel` / `GetMouseWheelV` / `GetMouseWheelH` / `GetMouseWheelEither`)

Queries vertical (`GetMouseWheel` / `GetMouseWheelV`), horizontal (`GetMouseWheelH`), or either-axis (`GetMouseWheelEither`, alias `GetMouseWheelAny`) scroll-wheel deltas for the current frame. Passing `consume = true` zeroes out `io.MouseWheel` / `io.MouseWheelH` so parent scrollable windows do not scroll while adjusting a hovered control.

```cpp
float clicks = ImGui::GetMouseWheelEither(true /* consume event */);
```

B. Continuous Range Chaser (`ImGui::AdjustItemOnMouseWheel`)

Attaches to **any** preceding continuous range item (`SliderFloat`, `SliderInt`, `DragFloat`, `DragInt`, etc.) via `ImGui::IsItemHovered()`. Overloaded for both `float*` and `int*`, it interprets vertical/horizontal mouse-wheel clicks, consumes the wheel event, and clamps boundaries.

- **Track-Only Cursor Precision:** Switches the cursor to East-West resize arrows (`ImGuiMouseCursor_ResizeEW` by default) strictly when hovering over the physical control track/box, preserving standard arrow cursors over text labels.

```cpp
#include "contrib/widgets/imgui_mouse_wheel_controls.h"

// Attach mouse-wheel adjustment to any standard ImGui slider or drag box:
ImGui::SliderFloat("Text Zoom", &fontZoom, 0.75f, 1.75f);
ImGui::AdjustItemOnMouseWheel(&fontZoom, 0.75f, 1.75f, 0.01f);

ImGui::DragInt("Altitude", &alt, 10.0f, 0, 40000);
ImGui::AdjustItemOnMouseWheel(&alt, 0, 40000, 100.0f);
```

C. Discrete Choice Chaser (`ImGui::CycleItemOnMouseWheel`)

Attaches to **any** preceding widget (`Button`, `Text`, `Combo`, `Selectable`) to step an integer index forward or backward one item per scroll gesture. Overloaded for both `(int* idx, int item_count, ...)` and `(int* idx, int min_idx, int max_idx, ...)`.

- **Clamped by Default (`wrap = false`):** Stops cleanly at the first and last items so a fast flick of the scroll wheel never accidentally wraps from minimum to maximum.
- **Directional Hover Cursor:** Defaults to `ImGuiMouseCursor_ResizeNS` (↕) on hover, and can be overridden per-call (e.g., `ImGuiMouseCursor_ResizeEW` ↔ for horizontal button cycling or `ImGuiMouseCursor_Hand`).

```cpp
// 1. Attach wheel-cycling to a popup-menu opener button with an East-West scroll cursor:
if (ImGui::ButtonLink(kDensityLabels[s_DensityIdx])) {
    ImGui::OpenPopup("##density_menu");
}
if (ImGui::CycleItemOnMouseWheel(&s_DensityIdx, 5 /* item_count */, false /* wrap */,
                                 true /* natural_list_order */, true /* consume */,
                                 ImGuiMouseCursor_ResizeEW)) {
    ApplyDensity(s_DensityIdx);
}

// 2. Standalone wheel-cycled ButtonLink paired with an abutting ResetButton:
//    Evaluating `(is_mod || can_restore) && static_cast<bool>(btn)` resets/restores on click
//    when actionable, and automatically falls back to click-to-toggle tooltip when idle at default!
const bool is_mod      = (s_DensityIdx != 2);
const bool can_restore = (!is_mod && s_LastDensityIdx != 2);
auto btn = ImGui::ButtonLink(kDensityLabels[s_DensityIdx], ImVec2(110.0f, 0.0f));
bool btn_clicked = (is_mod || can_restore) && static_cast<bool>(btn);
if (ImGui::CycleItemOnMouseWheel(&s_DensityIdx, 5, false, true, true, ImGuiMouseCursor_ResizeEW)) {
    if (s_DensityIdx == 2) s_LastDensityIdx = 2;
}

ImGui::SameLine(0.0f, 0.0f);
bool rst_clicked = ImGui::ResetButton("##density_reset", is_mod, 0.0f, true /* abutting */, nullptr, can_restore
if (btn_clicked || rst_clicked) {
    if (is_mod) { s_LastDensityIdx = s_DensityIdx; s_DensityIdx = 2; }
    else if (can_restore) { s_DensityIdx = s_LastDensityIdx; }
    ImGui::TimedTooltip::ClearActiveTooltip();
}
```

D. Turnkey Mouse-Wheel Sliders & Drag Boxes (`MouseWheelSlider` / `PercentSlider` / `MouseWheelDrag`)

Drop-in replacements for standard sliders and drag boxes that respond to mouse-wheel scrolling out of the box. `MouseWheelSlider` and `MouseWheelDrag` are overloaded for both `float*` and `int*` so you never have to type `Float` or `Int` in the function name!

```cpp
static float s_Heading = 180.0f;
static int   s_Fov     = 80;
ImGui::MouseWheelSlider("Heading", &s_Heading, 0.0f, 360.0f, "%.0f°", 0, 1.0f);
ImGui::MouseWheelSlider("FOV",     &s_Fov,     40,   120,    "%d°",   0, 1.0f);

static float s_PitchOffset = 0.0f;
ImGui::MouseWheelDrag("Pitch Offset", &s_PitchOffset, 0.5f /* speed & default wheel step */, -45.0f, 45.0f, "%.1
```

E. Composite Sliders & Drag Boxes with Reset & Undo (`SliderWithReset` / `PercentSliderWithReset` / `DragWithReset`)

Full-featured compound controls (overloaded for `float*` and `int*`, also aliased as `ResettableSlider`, `ResettablePercentSlider`, and `ResettableDrag`) with built-in mouse-wheel support, a flush abutting vector undo/reset button (`0`-gap seam), smart width reservation, hover isolation, active input lockout, and **Reset-with-Undo** (1-click A/B toggle between default and previous value).

- Sits cleanly in a single line: `[ Control Track/Box | ↺ / ↻ ] Label Text`
- **"Reset with Undo" (1-Click A/B Comparison):**
- When modified (`*v != v_default`), clicking the Reset button saves the current non-default value (`last_val`), resets `*v = v_default`, and flips the button into **Restore/Redo Mode** (↻ or `IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT`).
- Clicking the button again in Restore mode swaps `*v` right back to `last_val`, letting users click back and forth effortlessly to A/B compare their custom setting against the default!
- **Automatic Session Memory + Optional Persistent Pointer (`v_last`):** Works automatically with **zero extra arguments** via internal per-ID session memory (`##reset_undo`), or accepts an optional trailing `float* v_last = nullptr` / `int* v_last = nullptr` pointer if you want to persist the pre-reset value in your own variable/dataref across sessions.
- **Automatic 3-State Default Tooltips:** When `reset_tooltip` is `nullptr`, automatically formats the default and previous values using the control's `format` string:
- **Modified:** `"Click to revert to default (85%)."`
- **After Reset (Restore Mode):** `"At default (85%). Click to restore previous (42%)."`
- **At Default (Idle):** `"Already at default (85%)."` (Clicking the dimmed button or right-side label immediately toggles the status/help `TimedTooltip` ON/OFF!)
- **Dual Icon Configuration (`#define` or Runtime Setter) & Vector Fallback:**
- With zero font configuration, draws crisp, DPI-scaled procedural vector arrows directly in `ImDrawList`: counter-clockwise ↺ for Reset, and clockwise ↻ for Restore-Previous.
- Or configure custom FontAwesome / icon-font strings at compile time (`#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL ICON_FA_UNDO` and `#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT ICON_FA_REDO` / `IMGUI_CONTRIB_DEFAULT_REDO_LABEL`) or at runtime via `ImGui::SetDefaultResetButtonLabel(reset_label, restore_label)`. Button width is automatically sized across both glyphs so the control never shifts by 1px when toggling states.
- **Seamless Abutting Frame Borders:** When abutting an active control (`abutting == true`), `ResetButton` automatically suppresses redundant internal frame borders (`FrameBorderSize > 0.0f`), ensuring the control track and reset button share a single clean outer frame without an unwanted vertical box seam between them.

```cpp
static float s_Heading = 180.0f;
static int   s_Fov     = 80;

// Optional: configure custom Reset & Restore/Redo icons globally (or via #define before #include):
// ImGui::SetDefaultResetButtonLabel(ICON_FA_UNDO, ICON_FA_REDO);

// 1-line sliders (float* or int*) with built-in mouse-wheel and Reset-with-Undo:
ImGui::SliderWithReset("Heading", &s_Heading, 180.0f /* default */, 0.0f, 360.0f, "%.0f°");
ImGui::SliderWithReset("FOV",     &s_Fov,     80     /* default */, 40,   120,    "%d°");

static float s_Opacity = 0.85f;
ImGui::PercentSliderWithReset("Opacity", &s_Opacity, 0.85f /* default */, 0.1f, 1.0f);

static float s_PitchOffset = 0.0f;
ImGui::DragWithReset("Pitch Offset", &s_PitchOffset, 0.0f /* default */, 0.5f /* speed */, -45.0f, 45.0f, "%.1f°");
```

F. Turnkey Mouse-Wheel Combos (`MouseWheelCombo` / `ComboWithReset`)

Dropdown combo boxes that work like standard `ImGui::Combo` when clicked, and support live mouse-wheel cycling both when **closed** and while the dropdown popup menu is **open**!

- **Split Preview vs. Chevron `[▾]` Zones:**
- **Preview Text Zone (`[box_min.x .. box_max.x - GetFrameHeight()]`):** Shows `hover_cursor` (`ImGuiMouseCursor_ResizeNS` by default) and cycles items on mouse-wheel without falsely highlighting the right-hand chevron button.
- **Chevron Button Zone (`[box_max.x - GetFrameHeight() .. box_max.x]`):** Shows `ImGuiMouseCursor_Hand`, highlights strictly when the chevron itself is hovered (or when the popup is open), and **ignores mouse-wheel scrolling** so clicking/hovering the opener button feels like a dedicated dropdown button.
- **Configurable Chevron Arrow Polarity (`ImGui::ComboArrowStyle`):**

Configure globally via `#define IMGUI_CONTRIB_DEFAULT_COMBO_ARROW_STYLE` or `ImGui::SetDefaultComboArrowStyle(...)`:

- `ImGui::ComboArrowStyle_DownWhenClosed` (`0`, default): ▾ when closed $\rightarrow$ ▴ when open (or ▾ if the popup flipped upward near the bottom of the screen).
- `ImGui::ComboArrowStyle_RightWhenClosed` (`1`, ABC disclosure style): ▸ when closed $\rightarrow$ ▾ (or ▴ if opened upward) when open.
- `ImGui::ComboArrowStyle_UpWhenClosed` (`2`): ▴ when closed $\rightarrow$ ▾ (or ▴ if opened upward) when open.
- `ImGui::ComboArrowStyle_StaticDown` (`3`): Vanilla ImGui's static ▾.
- `ComboWithReset` (also aliased as `ResettableCombo`) pairs the mouse-wheel combo with the flush abutting ↺ / ↻ reset-with-undo button (with automatic session memory or optional `int* last_item = nullptr` persistence).
- **Per-Item Tooltips (`item_tooltips[]`) & Open-Menu Wheel Tooltip Latch ("Lens UI"):** Pass an optional array of tooltip strings (plain text or Markdown when `contrib.h` is active) to attach a distinct tooltip to each item in the open dropdown list. When the user spins the mouse wheel while the dropdown menu is open, tooltip targeting automatically latches onto the **newly wheel-selected item** (`*current_item`) rather than whichever row the stationary mouse cursor happens to be parked over, and seamlessly returns to normal mouse hover targeting when the mouse moves (`> 6px`).

```cpp
static const char* const kCameraModes[] = { "Cockpit", "Chase", "Circle", "Tower", "Ride-Along" };
static const char* const kCameraModeTooltips[] = {
    "**Cockpit View**: Inside the 3-D flight deck.",
    "**Chase View**: Locked behind the aircraft.",
    "**Circle View**: Free exterior orbit around the aircraft.",
    "**Tower View**: Stationary camera from the nearest ATC tower.",
    "**Ride-Along**: Wing-mount or fuselage camera position."
};
static int s_CamMode = 1; // Default: Chase (1)

// Optional: use disclosure chevron polarity (□ when closed -> ▼ when open):
// ImGui::SetDefaultComboArrowStyle(ImGui::ComboArrowStyle_RightWhenClosed);

// Standard mouse-wheel-enabled dropdown with per-item tooltips:
ImGui::MouseWheelCombo("Camera View", &s_CamMode, kCameraModes, IM_ARRAYSIZE(kCameraModes),
                       -1, false /* wrap */, kCameraModeTooltips);

// Compound mouse-wheel dropdown with abutting Reset-with-Undo (1 = Chase):
ImGui::ComboWithReset("Camera Mode", &s_CamMode, 1 /* default */,
                      kCameraModes, IM_ARRAYSIZE(kCameraModes),
                      -1, false /* wrap */, nullptr /* auto 3-state reset/restore tooltip */,
                      kCameraModeTooltips);
```

G. Mouse-Wheel Checkboxes (`MouseWheelCheckbox` / `ToggleItemOnMouseWheel` / `CycleItemOnMouseWheel(bool*)`)

Allows standard checkboxes to be toggled with the mouse wheel strictly when hovering over the **square `[✓]` box on the left** (`[box_min.x .. box_min.x + GetFrameHeight()]`), while hovering over the **text label** shows `ImGuiMouseCursor_Hand` (still clickable to toggle) and **ignores** the mouse wheel!

- By default (`wrap = false`), scrolling **up/right** checks the box (`true`) and scrolling **down/left** unchecks it (`false`), clamping cleanly at both ends (or pass `wrap = true` to flip on every wheel notch).
- Includes convenience overloads accepting `ImGuiMouseCursor_` directly after `bool* v` so you can customize or suppress the hover cursor (`ImGuiMouseCursor_Hand` or `ImGuiMouseCursor_None`) without passing intermediate boolean parameters.

```cpp
static bool s_AutoZoom = true;

// Option 1: Single-call MouseWheelCheckbox (with optional cursor override right after &v):
ImGui::MouseWheelCheckbox("Enable Auto-Zoom", &s_AutoZoom);
ImGui::MouseWheelCheckbox("Quiet Mode", &s_QuietMode, ImGuiMouseCursor_Hand);

// Option 2: 1-line chaser attached to standard ImGui::Checkbox:
ImGui::Checkbox("Expert Mode", &s_ExpertMode);
ImGui::ToggleItemOnMouseWheel(&s_ExpertMode, ImGuiMouseCursor_None); // or CycleItemOnMouseWheel(&s_ExpertMode)
```

H. Mouse-Wheel Auto-Defeat on Scrollable Viewports

In windows or child panels with active vertical scrolling (`ScrollMax.y > 0.0f`), spinning the mouse wheel to scroll through content could inadvertently adjust sliders, cyclers, or toggle switches if the mouse cursor passes over them.

To protect users against accidental setting changes while navigating long pages:

- **Automatic Detection:** `AdjustItemOnMouseWheel`, `CycleItemOnMouseWheel`, `MouseWheelSlider`, `PercentSlider`, `MouseWheelCombo`, `MouseWheelCheckbox`, and `ToggleButton` inspect the active Dear ImGui window and parent hierarchy for vertical scrollability via `ImGui::Contrib::IsCurrentWindowScrollableY()`.
- **Zero Accidental Adjustments:** When vertical scrolling is active, in-widget mouse-wheel adjustments are **defeated automatically**, and wheel events pass directly through to window scrolling.

- **Smart Cursor Hiding:** Resizing/adjustment hover cursors (`ImGuiMouseCursor_ResizeEW`, `ImGuiMouseCursor_ResizeNS`) are suppressed on scrollable viewports so users are not misled into expecting wheel adjustments.
- **Per-Widget Override:** If you explicitly want mouse-wheel interaction on a widget within a scrollable panel (e.g. an embedded canvas or dedicated map zoomer), pass `bAllowMouseWheelOnScrollable = true` to the widget overload.

---

3. Clickable Link & Item Cursor Affordances (`imgui_clickable_links.h`)

In standard Dear ImGui, buttons, menu items, and clickable text keep the default OS arrow cursor. The `imgui_clickable_links.h` header eliminates the boilerplate of writing `if (IsItemHovered()) SetMouseCursor(ImGuiMouseCursor_Hand);` on every interactive item, while providing automatic **click-to-toggle tooltip** ergonomics and **open-popup wheel-selection tooltip latching**.

- **Ambient Color Preservation on `ImGui::TextLink`:** In Dear ImGui 1.91+, the built-in `ImGui::TextLink(const char*)` forces `ImGuiCol_TextLink` (a bright sky-blue tint) with an underline. Our `imgui_clickable_links.h` overloads preserve your **ambient** `ImGuiCol_Text` **color** by default (or accept an explicit `const ImVec4* col_override` and `std::string`), while rendering a zero-frame clickable text item with a Hand cursor on hover (with Dear ImGui's native hyperlink preserved as `ImGui::NativeTextLink`).
- **Smart Click-to-Toggle Tooltips (`ImGui::LinkClickResult`):** `ButtonLink`, `SmallButtonLink`, `TextLink`, `TextLinkUnformatted`, `MenuItemLink`, and `MD::TextLink` return `ImGui::LinkClickResult`:
- **Tested in `if (...)` (Action Link):** Evaluating `if (ImGui::ButtonLink("Save"))` invokes `operator bool()`, marking the click as handled by your action block and dismissing any open tooltip on click.
- **Standalone Statement (Info / Help Link):** Calling `ImGui::TextLink("(i) Help");` without testing its return value leaves the click unconsumed, so clicking the widget immediately toggles a trailing `ImGui::TimedTooltip::Text` / `TextMD` ON and OFF (without closing any enclosing `BeginPopup`!).

```cpp
#include "contrib/widgets/imgui_clickable_links.h"

// 1. Action button (tested in `if` -> executes action & dismisses tooltip on click):
if (ImGui::ButtonLink("Save Flight Plan")) {
    SavePlan();
}
ImGui::TimedTooltip::TextMD("Saves the active flight plan to disk.");

// 2. Standalone info link (return value ignored -> clicking toggles tooltip ON/OFF!):
ImGui::TextLinkUnformatted("(i) Works with scroll-wheel!");
ImGui::TimedTooltip::TextMD("Spin the **scroll-wheel** over this menu to step through options.");

// 3. Checkboxes, radio buttons, selectables, and menu items with hand cursor:
ImGui::CheckboxLink("Enable Autothrottle", &s_Autothrottle);
ImGui::RadioButtonLink("NAV 1", &s_NavSource, 1);
ImGui::SelectableLink("Row Item", isSelected);
ImGui::MenuItemLink("Compact Density", nullptr, isSelected);
```

Custom `BeginPopup` Menus with Mouse-Wheel Cycling & Per-Item Tooltips

When building custom `ImGui::BeginPopup` menus (with custom titles, separators, or footers), wrap the popup contents in `BeginGroup()` / `EndGroup()`, use `ImGui::MenuItem` (or `ImGui::MenuItemLink` for a Hand cursor on each row) with per-item `TimedTooltip::TextMD` calls, and attach `ImGui::CycleItemOnMouseWheel` after `EndGroup()`:

```cpp
if (ImGui::BeginPopup("#density_menu")) {
    ImGui::BeginGroup();
    if (ImGui::TextLink("Choose tracking list density:")) {
        ImGui::CloseCurrentPopup();
    }
    ImGui::Separator();

    for (int i = 4; i >= 0; --i) {
        bool isActive = (densityIdx == i);
        if (ImGui::MenuItem(menus[i], nullptr, isActive)) {
            gUserLinePadding = 4 + 2 * i;
        }
        ImGui::TimedTooltip::TextMD("Switch table spacing to **%s** density.", tipLabels[i]);
    }

    ImGui::Separator();
    ImGui::TextLinkUnformatted("  (i) Works with scroll-wheel!  ");
    ImGui::TimedTooltip::TextMD("Spin your scroll-wheel over this list to step through options.");
    ImGui::EndGroup();

    // Whole-menu mouse-wheel cycling (automatically latches per-item tooltips to `isActive`!):
    if (ImGui::CycleItemOnMouseWheel(&densityIdx, 0, 4, /*wrap=*/false, /*natural_list_order=*/false,
                                     /*consume=*/true, ImGuiMouseCursor_Hand)) {
        gUserLinePadding = 4 + 2 * densityIdx;
    }
    ImGui::EndPopup();
}
```

- **Automatic Wheel-Selection Tooltip Latch:** **`ImGui::MenuItem`** and **`ImGui::MenuItemLink`** register each item's **`isActive`** state. When the user spins the scroll wheel inside the open popup, **`TimedTooltip`** automatically switches from mouse-hover targeting to showing the tooltip for the **newly wheel-selected item** (**`isActive`**), returning to normal hover targeting as soon as the mouse moves (**`> 6px`**).

Universal Item Hover-Cursor Chaser (`ImGui::AttachItemCursor`):

```cpp
// Attach default Hand cursor to any preceding item:
ImGui::Image(textureId, ImVec2(64, 64));
ImGui::AttachItemCursor();

// Or attach any specific ImGuiMouseCursor on hover:
ImGui::Button("Splitter");
ImGui::AttachItemCursor(ImGuiMouseCursor_ResizeEW);
```

4. Badges, Pills & Key Caps (`imgui_badges.h`)

Sleek, standalone C++ UI indicators designed for status tags, version numbers, notification counters, and keyboard shortcut chips without requiring any Markdown parsing.

```cpp
#include "contrib/widgets/imgui_badges.h"

// 1. Semantic Status Badges (Default, Primary, Success, Warning, Danger, Info):
ImGui::Badge("ONLINE", ImGui::ImGuiBadgeVariant_Success);
ImGui::SameLine();
ImGui::Badge("EXPIRING", ImGui::ImGuiBadgeVariant_Warning);
ImGui::SameLine();
ImGui::Badge("FAULT", ImGui::ImGuiBadgeVariant_Danger);

// 2. Fully-rounded Pills:
ImGui::Pill("v1.92", ImGui::ImGuiBadgeVariant_Primary);
ImGui::SameLine();
ImGui::Pill("PRO", ImVec4(0.85f, 0.40f, 0.10f, 1.0f)); // Custom ImVec4 background

// 3. Custom Hex or ImU32 Backgrounds with Automatic Contrast Calculation:
ImGui::Badge("BETA", ImGui::GetColorU32(ImVec4(0.9f, 0.9f, 0.1f, 1.0f))); // Auto-selects dark text

// 4. Keyboard Key Caps (<kbd>):
ImGui::Text("Press");
ImGui::SameLine();

ImGui::Kbd("Ctrl+Shift+P");
ImGui::SameLine();
ImGui::Text("to open Command Palette.");
```

- **Features:**
- **Pixel-Balanced Rounded Geometry:** Uses TrueType baseline alignment and font-scale compensation so ascenders and descenders ('p', 'g', 'y') stay centered with identical visual margins.
- **Intelligent Auto-Contrast:** Computes ITU-R BT.601 perceptual luminance of the background color and automatically selects crisp dark (**`#000000`**) or light (**`#FFFFFF`**) text when foreground color is omitted.
- **3D Key Caps (`ImGui::Kbd`):** Renders tactile keyboard shortcut chips with rounded borders and a bottom bevel shadow.
- **Zero Markdown Overhead:** Standalone immediate-mode C++ functions usable anywhere in pure ImGui panels. Markdown's **`<badge>`**, **`<pill>`**, and **`<kbd>`** tags delegate directly to these widgets.

---

5. Procedural Spinners & Countdown Dials (`imgui_spinners.h`)

Smooth, procedural immediate-mode activity indicators and radial timers rendered directly via **`ImDrawList`** antialiased paths. No texture assets, no font hacks, and zero memory allocations.

```cpp
#include "contrib/widgets/imgui_spinners.h"

// 1. Organic Activity Spinner (loading / busy state):
ImGui::Spinner("##loading"); // Auto-scales to font size
ImGui::SameLine();
ImGui::Text("Syncing telemetry...");

// With custom radius, thickness, color, and label:
ImGui::Spinner("Downloading airport navdata", 14.0f, 3.0f, IM_COL32(0, 200, 255, 255));

// 2. Radial Countdown Timer (fraction remaining: 0.0f .. 1.0f):
float timeoutRemaining = GetRemainingFraction(); // e.g., 0.65f
ImGui::RadialTimer("Auto-dismiss in 5s", timeoutRemaining);

// 3. Countdown Timer Helper (current seconds vs max duration):
float alertTimeLeft = 3.2f;
float alertMaxTime  = 10.0f;
ImGui::CountdownTimer("Alert Timeout", alertTimeLeft, alertMaxTime);
```

- **Features:**
- **Organic Breathing Arc Animation:** Rather than a rigid rotating stick, **`ImGui::Spinner`** uses a sinusoidal expansion-and-contraction cycle that mimics modern Material/Apple loading spinners.

- **Clockwise/Counter-Clockwise Dials:** `ImGui::RadialTimer` starts cleanly at 12 o'clock (`-90°`) with an ambient background track ring, sweeping smoothly around to indicate elapsed or remaining time.
- **True Ambient Scaling:** Radius and stroke thickness automatically scale to the ambient font size (`ImGui::GetFontSize()`) when left as `0.0f`, looking sharp on standard and high-DPI displays alike.
- **Built-in Text Label Affordance:** Labels without leading `##` automatically render beside the spinner or dial with standard item alignment and spacing.

---

6. Universal Standalone & Chaser Copy Buttons (`imgui_copy_button.h`)

Tactile, immediate-mode copy buttons featuring procedural vector icons, transient "Copied!" feedback, anti-twitch fixed width layouts, and seamless integration with `TimedTooltip` and `InputText`.

```[CPP]
#include "contrib/widgets/imgui_copy_button.h"

// 1. Standalone Proxy Form (CopyButton / CopyBufferButton):
// Takes an explicit string, buffer, or std::string to copy to the clipboard:
ImGui::CopyButton("##copy_apikey", myApiKeyString);
// Or use the explicit semantic alias:
ImGui::CopyBufferButton("##copy_buf", textBuffer);

// 2. Chaser Form (CopyLastItemButton / CopyTextFieldButton):
// Placed immediately after an ImGui::InputText / InputTextMultiline:
static char s_LogBuffer[512] = "N450GA clearance delivered...";
ImGui::InputText("Flight Plan", s_LogBuffer, sizeof(s_LogBuffer));
ImGui::SameLine();
ImGui::CopyLastItemButton("##copy_flightplan"); // Automatically copies s_LogBuffer!

// 3. Querying transient state or custom duration:
bool is_copied = false;
if (ImGui::CopyButtonEx("##custom_copy", "Payload", 2.0f /* duration */, ImVec2(0, 0), &is_copied)) {
    // Button was clicked this frame!
}
```

- **Features:**
- **Procedural Vector Icons:** Renders a clean two-sheet overlapping clipboard vector icon that transitions smoothly into a green vector checkmark during the transient `Copied!` confirmation state.
- **Anti-Twitch Fixed Width Layout:** Automatically calculates the maximum bounding width across `"Copy"` and `"Copied!"` labels (`ImMax(normal_w, copied_w)`) so adjacent buttons and toolbars never twitch or jump when toggling states.
- **Independent `ImGuiStorage` Scoping:** Transient copied feedback is keyed to `ImGuiID` in the window's state storage, ensuring multiple copy buttons in the same window maintain their own timers and feedback independently.
- **Zero-Redundancy Chaser Mode (`CopyLastItemButton`):** Automatically queries Dear ImGui's internal `LastItemData` and `InputTextState` to retrieve the buffer pointer of the preceding input field with zero redundant buffer duplication.
- **Semantic Aliases:** Includes `ImGui::CopyBufferButton` and `ImGui::CopyTextFieldButton` to make developer intent clear at a glance.
- **Rich `TimedTooltip` Integration:** Automatically displays `"Copy to Clipboard"` on hover, switching to `"Copied!"` during the feedback duration, with an automatic `ImGuiMouseCursor_Hand` hover cursor.

---

7. Avionics Rotary Knobs & Dimmer Dials (`imgui_knobs.h`)

Turnkey immediate-mode cockpit potentiometers, rotary dials, and lighting dimmer controls featuring an unmistakable 270° "Omega" arc, tactile two-step `OFF` detente switches, perimeter graduation ticks, and full mouse-wheel integration.

```cpp
#include "contrib/widgets/imgui_knobs.h"

// 1. Avionics Potentiometer Dimmer with 2-Step OFF Detente:
// Turning CCW dims smoothly from 100% down to 5% (minimum active floor).
// An intentional extra downward notch while resting at 5% snaps across into "OFF"!
// Turning CW from OFF snaps immediately back into active range (5%).
static float s_FloodDimmer = 0.85f;
ImGui::DimmerKnob("Floodlight Dimmer", &s_FloodDimmer, 0.05f /* v_min_active */, 1.0f /* v_max */, 1.0f /* defau

// 2. Standard 270° Rotary Knob with 1-Click Vector Reset (□):
static float s_RadioVol = 0.70f;
ImGui::KnobWithReset("Radio Volume", &s_RadioVol, 0.70f /* default */, 0.0f, 1.0f, "%.0f%%", 0.0f, 0.02f, "Reset

// 3. Compact / Percentage Dial:
static float s_HeadingBug = 180.0f;
ImGui::Knob("Heading Bug", &s_HeadingBug, 0.0f, 360.0f, "%.0f°", 14.0f /* radius */);

// 4. Vertical Stacked Avionics Gauge Unit:
static float s_TransponderGain = 0.50f;
ImGui::Knob("XPNDR", &s_TransponderGain, 0.0f, 1.0f, "%.2f", 20.0f, 0.01f, ImGuiKnobFlags_ValueBelow | ImGuiKnob
```

- **Features:**
- **Unmistakable Omega Potentiometer Geometry:** Features an authentic 270° arc with a 90° open deadband at 6 o'clock and a raised 3D knurled knob hub with a high-contrast rotating index needle/pip. Instantly disambiguated from 360° read-only timers or progress circles.
- **Tactile Two-Step Detente Switch:** Counter-clockwise rotation stops cleanly at the minimum active floor (e.g. 5%), preventing accidental power cutoffs in flight. A deliberate extra scroll step past the stop snaps across the detente gap directly into the powered-down **`OFF`** detente. Clockwise rotation from **`OFF`** snaps immediately back to minimum power.
- **State-Aware Text Readout:** Automatically displays **`"OFF"`** in warning amber when disarmed in detente, or the formatted percentage/value when active.
- **Dual-Mode Input Ergonomics:** Primary mouse-wheel rotation with automatic scroll-defeat prevents scrolling the parent window. Secondary vertical/horizontal click-dragging supports VR/touchscreen use, with **`Shift`** (fine $0.1\times$) and **`Ctrl`** (coarse $2.0\times$) speed modifiers.
- **Default `ResizeEW` (<-->) Cursor:** Shows horizontal range selection cursor by default on hover, confirming interactive status at a glance.
- **Double-Click Reset / Detente Toggle:** Double-clicking the knob restores calibrated baseline default (or toggles between **`OFF`** and default).

---

8. Directional Popup Menus & Select Pulldowns (`imgui_popup_menus.h`)

First-class popup anchoring and selection widgets solving directional placement, muscle-memory toggle protection, font-scale invariant alignment, and closed-state wheel pass-through:

```cpp
#include "contrib/widgets/imgui_popup_menus.h"

// 1. Directional Item Anchoring (Status Bars & HUDs):
// Blossoms UP above the button with sub-pixel precision:
if (ImGui::Button("View Mode")) {
    ImGui::OpenPopupAtItem("##ViewModeMenu", ImGuiPopupAnchor_AboveItem, ImGuiPopupAlign_Auto);
}

// 2. Begin Anchored Popup:
if (ImGui::BeginAnchoredPopup("##ViewModeMenu")) {
    if (ImGui::MenuItem("Circle View")) { SetView(0); }
    if (ImGui::MenuItem("Spot View"))   { SetView(1); }
    if (ImGui::MenuItem("Tower View"))  { SetView(2); }
    ImGui::EndPopup();
}

// 3. Headroom Query for Window Managers / Subclasses (e.g. WidgetImgWindow):
// Returns exact measured height in pixels so XPLM window expansion has zero guessing:
float headroom = ImGui::GetLastPopupHeight("##ViewModeMenu");

// 4. SelectPulldown Widget (Settings Window 1-Line Dataref Selectors):
// Closed: ignores mouse-wheel (smooth parent list scrolling without accidental edits).
// Open: mouse-wheel cycles active selection dynamically without closing menu!
// Caret: automatically renders crisp procedural vector arrows (▼ / ▲) without requiring Unicode font glyphs!
// Lens UI: stationary wheel spinning latches rich tooltips directly to active selection!
static int s_SelectedCam = 1;
const char* const kCamModes[] = { "Circle", "Spot", "Tower", "Cockpit" };
const char* const kCamTips[]  = { "**Circle**\nFollow aircraft", "**Spot**\nFixed distance", "**Tower**\nTower v
if (ImGui::SelectPulldownMD("##CameraMode", &s_SelectedCam, kCamModes, 4, kCamTips, 0.0f, ImGuiPopupAnchor_Below
    UpdateCamera(s_SelectedCam);
}

// 5. Upward Blossoming Pulldown (Status Bars & Screen-Bottom Docks):
static int s_PowerMode = 0;
const char* const kPowerModes[] = { "Battery", "APU", "External Power" };
if (ImGui::SelectPulldown("##PowerMode", &s_PowerMode, kPowerModes, 3, nullptr, 140.0f, ImGuiPopupAnchor_AboveIt
    UpdatePowerSource(s_PowerMode);
}

// 6. Caret Customization & Procedural Vector Drawing:

// Optional: Configure FontAwesome glyphs or custom ASCII strings globally at runtime:
ImGui::SetDefaultPulldownCaret(ICON_FA_CHEVRON_DOWN, ICON_FA_CHEVRON_UP);

// Or call the procedural vector arrow helper directly in custom controls:
ImGui::DrawPulldownArrow(ImGui::GetWindowDrawList(), arrow_pos, text_col, ImGuiDir_Down, font_size);
```

- **Features:**
- **"Lens UI" Selection-Latched Tooltips:** When spinning the mouse wheel inside an open pulldown menu, per-item tooltips dynamically latch onto the newly selected item (`*current_item`) instead of remaining frozen over whichever row the stationary cursor rests on. Deliberate mouse movement (`> 6 px`) cleanly breaks the latch and reverts to standard spatial hover.
- **Predictable Wheel Stepping & Circular Wrapping:** Supports optional `bool wrap = false` and `bool natural_list_order = true` parameters. When `wrap = true`, spinning past the ends wraps around smoothly; `natural_list_order = true` advances down the list on downward scroll (`clicks < 0.0f`).
- **Automatic Focus & Scroll-Into-View:** Wheel-selected items receive `SetItemDefaultFocus()` and automatically scroll into view (`SetScrollHereY()`) in long scrollable pulldowns.
- **Dynamic Cursor Cues:** Opener button displays `ImGuiMouseCursor_Hand` on hover; open menu options and background display `ImGuiMouseCursor_ResizeNS` to visually signal mouse-wheel responsiveness.
- **Rich Markdown Tooltip Aliases (`SelectPulldownMD` / `ImGui::MD::SelectPulldown`):** Full support for rich Markdown tooltips (`TimedTooltip::MD::TextProxy`), available via root and sub-namespace aliases.
- **Procedural Vector Caret Fallback:** Eliminates unmapped Unicode Geometric Shapes (`U+25BE`) missing-glyph issues. Defaults to crisp, antialiased procedural vector triangles rendered directly to `ImDrawList`, matching `ResetButton`

vector rendering.

- **Configurable Font Carets & Runtime Setters:** Host apps with FontAwesome can define `#define IMGUI_CONTRIB_DEFAULT_PULLDOWN_CARET_DOWN` or call `ImGui::SetDefaultPulldownCaret(...)` to use native icon-font glyphs.
- **Directional Blossoming (`ImGuiPopupAnchor`):** Supports `ImGuiPopupAnchor_AboveItem` for bottom status bars and docks, dynamically flipping caret directions between open and closed states.
- **Smooth TimedTooltip Integration:** Item hover tooltips automatically use `ImGui::TimedTooltip` (with fallback to `ImGui::SetTooltip`), preventing cursor jitter.
- **Broad In-Popup Hover Scope:** Mouse-wheel spinning anywhere inside the open popup window (including while hovering individual `Selectable` rows) cycles active options smoothly.
- **Font-Scale Invariant Placement:** Uses live `GetItemRectMin()` and `ItemSpacing.y` bounding geometry from the rendering frame, eliminating magic pixel offsets and text-zoom drift.
- **Muscle-Memory Toggle Safety:** Clicking the opener button while the popup is open cleanly toggles it closed without re-opening flicker.
- **Universal Widget Compatibility:** `OpenPopupAtItem()` anchors cleanly behind any widget: `ImGui::Button`, `ImGui::ButtonLink`, `ImGui::MD::ButtonMD`, `ImGui::Pill`, `ImGui::Badge`, or custom controls.
- **Exact Headroom Reporting:** `GetLastPopupHeight(str_id)` reports the measured pixel height for dynamic XPLM window expansion.
- **Scroll Pass-Through:** `SelectPulldown` never captures the mouse-wheel while closed, preventing unwanted dataref changes while scrolling settings.

---

10. Procedural Vector Icons Engine (`imgui_icons.h`)

A zero-texture, zero-font, zero-dependency procedural vector icon engine that actively renders crisp antialiased vector geometry directly to Dear ImGui's `ImDrawList`.

Completely eliminates missing-glyph diamond replacement characters (`) when host font atlases only bake standard ASCII/Latin-1 (0x20..0xFF) and lack extended Unicode blocks (such as Mathematical arrows U+2190..U+21FF or Geometric shapes U+25A0..U+25FF`).

[CPP]

```cpp
#include "contrib/widgets/imgui_icons.h"

// 1. Direct Immediate-Mode Widget:
// Scales automatically with ambient font size (GetFontSize() * 0.90f) and centers on text baseline:
ImGui::Icon("undo");
ImGui::Icon("caret-up", 18.0f);
ImGui::Icon("gear", 16.0f, &ImVec4(1.0f, 0.6f, 0.0f, 1.0f));

// 2. Procedural Vector Push-Buttons:
// IconButton: square button with centered vector icon, hand cursor, and optional tooltip:
if (ImGui::IconButton("##zoom_up", "caret-up", ImVec2(0, 0), "Zoom In")) {
    ZoomIn();
}

// ButtonIcon: dual icon + text button (or text + icon if icon_on_right = true):
if (ImGui::ButtonIcon("Settings", "gear", ImVec2(0, 0), false, "Open avionics configuration")) {
    OpenSettings();
}
if (ImGui::ButtonIcon("Next Page", "caret-right", ImVec2(0, 0), true)) {
    AdvancePage();
```

```
}

// 3. Direct DrawList Procedural Vector Path Rendering:
ImGui::DrawVectorIcon(ImGui::GetWindowDrawList(), pos, size, col_u32, "search");

// 4. Forgiving Name Normalization:
// Accepts any format: "undo", ":undo:", "ICON_FA_UNDO", "ICON_UNDO", "reset", or raw UTF-8:
ImGui::Icon("ICON_FA_UNDO");
ImGui::Icon(":gear:");
ImGui::Icon("\xE2\x86\xBA"); // □
```

Markdown & Typography Integration

The icon engine is deeply integrated into **`ImGui::MarkdownExt`** and **`ImGui::TextStyled`**, offering 5 authoring modes:

1. **Inline HTML Tag:** **`<icon caret-up />`**, **`<icon orange undo />`**, **`<icon name=gear size=16 />`**
2. **Emoji Shortcodes:** **`:undo:`**, **`:reset:`**, **`:caret_up:`**, **`:caret-down:`**, **`:gear:`**, **`:check:`**, **`:star:`**, **`:plane:`**
3. **Steven's Raw-Fallback Link Macro:** **`#[^](ICON_CARET_UP)`** or **`#[↺](ICON_UNDO)`**
4. **Automatic Raw Unicode Glyph Rescue:** Raw symbols like ↺, ↻, ▲, ▼, ✓, ✲ outside code blocks are automatically intercepted and rendered as crisp vector icons with zero missing-glyph diamonds!
5. **Backtick Rescue:** Single symbols wrapped in backticks (`` ↺ ``) are rescued and rendered as vector icons.

Universal Icon Catalog

- **Navigation:** **`caret-up`** (▲), **`caret-down`** (▼), **`caret-left`** (◀), **`caret-right`** (▶)
- **History:** **`undo`** / **`reset`** (↺), **`redo`** / **`restore`** (↻)
- **Actions & State:** **`check`** (✓), **`cross`** (✗), **`plus`** (+), **`minus`** (-), **`gear`** (✲), **`search`** (■)
- **Indicators & Badges:** **`info`** (ℹ), **`warning`** (■), **`star`** (★), **`plane`** (✈), **`circle`** (●), **`square`** (■)

Credits & Upstream Lineage

- **Author & Architect**: Steven L. Goldberg (@slgoldberg)
- **SliderPercent Upstream Inspiration**: Birger Hoppe (LiveTraffic)
- **License**: BSD 3-Clause (see ImgWindow LICENSE)